

2026 비전공 교원 대상 AI 특강 (하계)

<기계학습 딥러닝 분야>

실습: 2026년 8월 5일 AutoML
소프트웨어학과 김광수 교수
인공지능학과 장승우 박사과정

AutoML

환경 설정

AutoML — 환경 설정

실습 환경 — 슈퍼컴퓨팅센터 GPU 클러스터

성균관대학교 슈퍼컴퓨팅센터

- 학교가 보유한 고성능 컴퓨팅(HPC) 자원을 교내·외 연구자에게 클라우드 형식으로 제공
- GPU 서버: NVIDIA A100(80GB)

접속 및 사용 방식

- 웹 브라우저로 접속 — 내 노트북에 아무것도 설치하지 않음
- 각자에게 도커 컨테이너 자동 배정
- 내 컴퓨터 사양과 무관 — 무거운 학습은 서버의 GPU가 대신 계산

구분	내부 사용자	외부 사용자
1 GPU	1,600원	3,200원

* 요금제 적용기간: 슈퍼컴퓨팅센터 운영위원회에서 결정

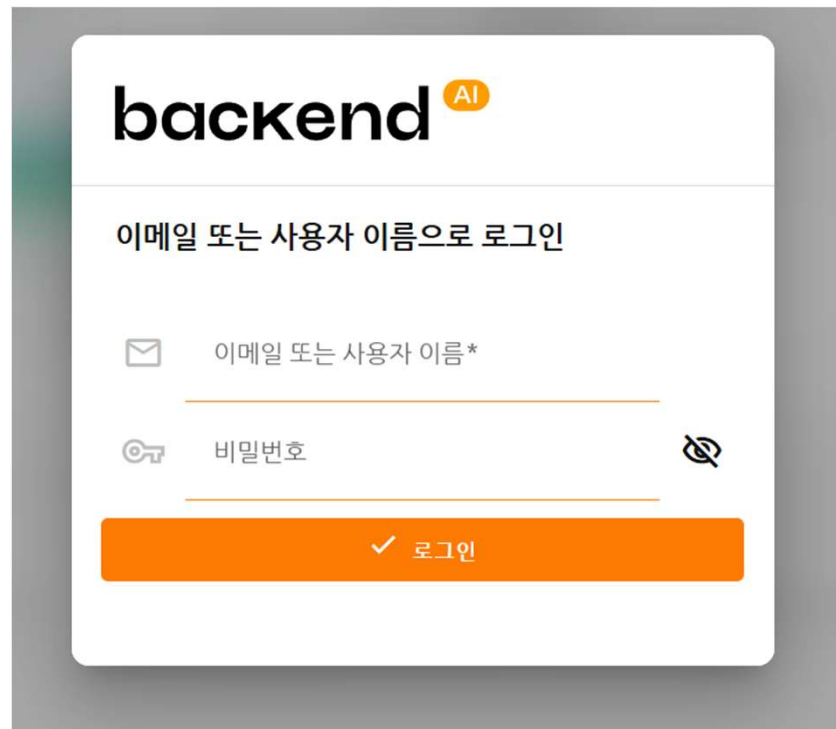
* 각 금액은 1시간 기준 요금임. 사용량 계산시 15분 단위 절사

* 통화는 KRW 기준이며, VAT미포함

<https://supercom.skku.edu/supercom/index.do>

AutoML — 환경 설정

실습 환경 — 슈퍼컴퓨팅센터 GPU 클러스터

A login form for 'backend AI'. The form has a title 'backend AI' with 'AI' in an orange circle. Below the title is the instruction '이메일 또는 사용자 이름으로 로그인'. There are two input fields: the first is labeled '이메일 또는 사용자 이름*' and the second is labeled '비밀번호' with a toggle icon. At the bottom is an orange button with a checkmark and the text '로그인'.

아이디: 본인의 SKKU 이메일 주소

비밀번호: 이메일 아이디의 @ 앞 부분 + !

sewo@skku.edu → sewo!

<https://spccluster.skku.edu/start>

실습 환경 — 슈퍼컴퓨팅센터 GPU 클러스터

2. VPN 접속(윈도우 PC, MAC, Mobile(Android, iOS))

- 1) <https://vpn.skku.edu:9000> 접속 (최초 접속 시 클라이언트 프로그램 다운로드)
- 2) 다운로드 된 클라이언트 프로그램 설치
- 3) 다운로드된 VPN 프로그램(AXGATE VPN) 실행
 - ☐ ID/PW : KINGO ID/PW로 로그인
- 4) OTP 등록(OS별 접속가이드 참조하여 구글 Authenticator 또는 AXGATE OTP 사용하여 QR 등록)
- 5) OTP 입력 후 사용

(※ VPN 사용은 윈도우 PC 사용을 권장합니다)

3. VPN 접속가이드(윈도우 PC, MAC, Mobile(Android, iOS), Linux)

- ☐ PC(Windows)용 접속 가이드 : [\[다운로드 링크\]](#) // [\[English Download\]](#)
- ☐ Mac용 접속 가이드 : [\[다운로드 링크\]](#)
- ☐ Android용 접속 가이드: [\[다운로드 링크\]](#)
- ☐ iOS용 접속 가이드: [\[다운로드 링크\]](#)
- ☐ Linux용 접속 가이드 : [\[다운로드 링크\]](#) / Linux용 접속 프로그램 : [\[다운로드 링크\]](#)
- ☐ OTP Registration Guide (Eng) : [\[Download\]](#)

<https://vpninfo.skku.edu/>

AutoML — 환경 설정

실습 환경 — 슈퍼컴퓨팅센터 GPU 클러스터

The screenshot displays the backend.ai webUI interface. The top navigation bar is orange and contains the 'backend.ai WebUI' logo, a '프로젝트' dropdown menu set to 'Edu', and user information '장승우'. A left sidebar lists navigation options: 시작 (Start), 대시보드 (Dashboard), 스토리지 (Storage), 데이터 (Data), 워크로드 (Workload), 세션 (Session), 나의 실행 환경 (My Execution Environment), 메트릭 (Metrics), 자원 요약 (Resource Summary), and 통계 (Statistics). The main content area features a yellow notification banner at the top stating: '비밀번호를 변경해 주세요. 상단바의 사람 아이콘을 클릭하고 "사용자 정보 변경" 메뉴로 이동하세요.' Below this, a status box for 'SKKU HPC Cluster [backend.ai]' indicates a '-Pending status means 'entered the queue'' and mentions '신규 최신 GPU(Blackwell) 연산 노드 하반기 도입 예정'. At the bottom, four white cards with orange icons offer actions: '새 스토리지 폴더 생성하기' (Create new storage folder), 'Interactive 세션 생성' (Create Interactive session), 'Batch 세션 생성' (Create Batch session), and 'URL 로 시작' (Start with URL). Each card includes a brief description and an orange button to proceed.

backend.ai WebUI

프로젝트 Edu

장승우

시작 /

SKKU HPC Cluster [backend.ai]

-Pending status means 'entered the queue'

신규 최신 GPU(Blackwell) 연산 노드 하반기 도입 예정

새 스토리지 폴더 생성하기

폴더를 만들고 파일을 업로드합니다. 모델을 학습시키거나 외부 서비스를 제공하려면 해당 작업은 필수적입니다.

폴더 생성하기

Interactive 세션 생성

모델을 학습시킬 세션을 생성합니다. 원하는 환경과 리소스를 선택하여 코드를 실행하세요.

세션 시작

Batch 세션 생성

이미 정의된 파일 또는 예약된 작업에 대한 배치 세션을 생성합니다. 명령을 입력하고 날짜와 시간을 설정한 다음 필요에 따라 세션을 실행하세요.

세션 시작

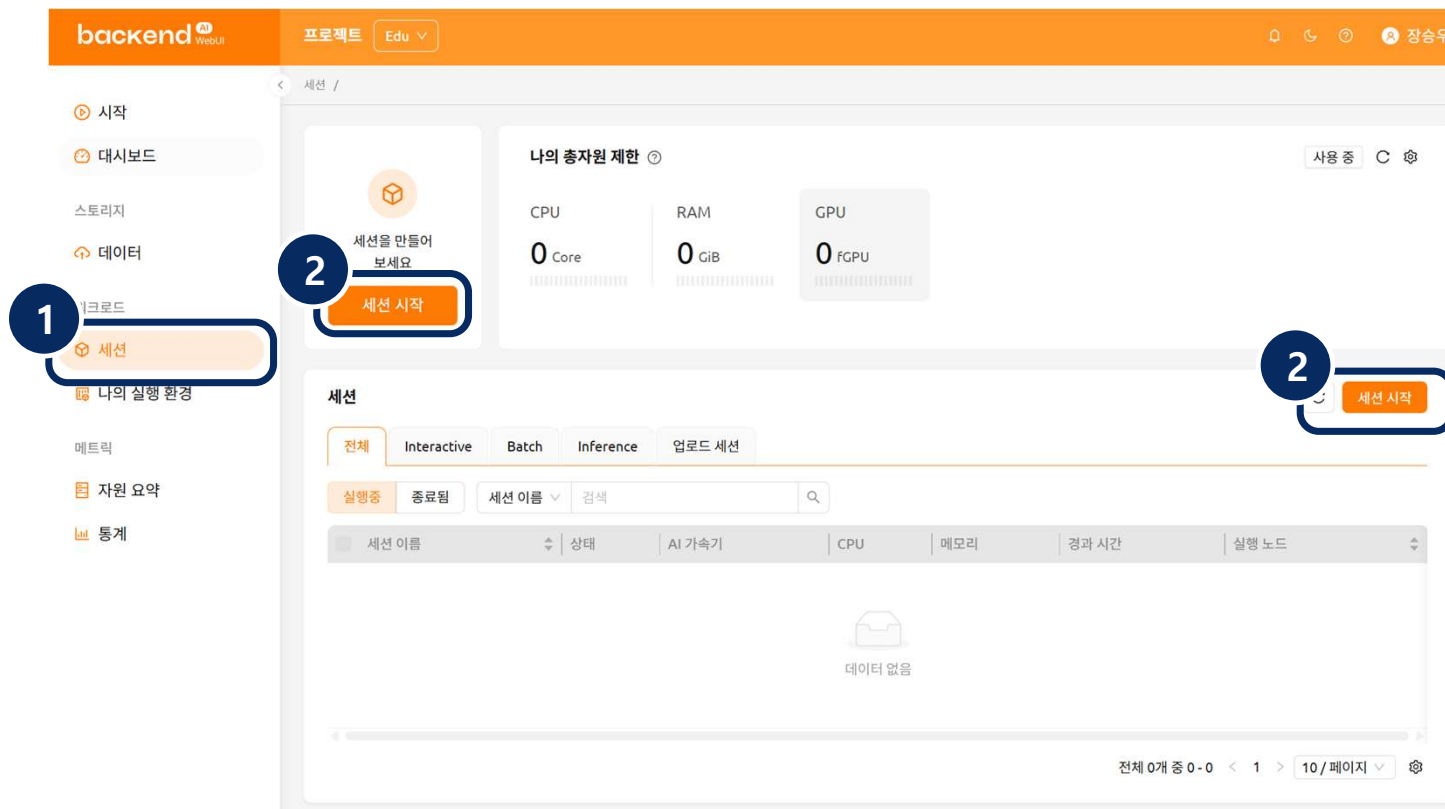
URL 로 시작

GitHub, GitLab, Notebook 등 다양한 환경에서 프로젝트와 코드를 가져오고, 실행할 수 있습니다.

지금 시작하기

AutoML — 환경 설정

실습 환경 — 슈퍼컴퓨팅센터 GPU 클러스터



실행 순서

- ① 왼쪽 메뉴에서 '세션' 클릭
- ② '세션 시작' 버튼 클릭

세션: 사용자가 모델 학습, 데이터 분석 등을 실행할 수 있는 독립적인 컴퓨팅 환경

AutoML — 환경 설정

실습 환경 — 세션 타입

새 세션 시작

최근 기록

1 세션 타입

2 실행 환경 & 자원 할당

3 데이터 & 폴더

4 네트워크

▶ 검토 및 시작

세션 타입

☒ **Interactive** 주피터 노트북, 비주얼 스튜디오 코드 등을 통해 코드를 인터랙티브하게 생성, 테스트, 실행할 수 있게 해줍니다.

☐ **Batch** 여러 노드 클러스터로 코드를 실행합니다.

세션 이름 (선택)

초기화

다음 >

검토로 건너뛰기 >>

↑
[다음] 클릭

AutoML — 환경 설정

실습 환경 — 실행 환경 & 자원 할당

새 세션 시작

최근 기록

실행 환경

실행 환경 / 버전

PyTorch (NGC)

repo

NVIDIA GPU Cloud

24.09

x86_64

PyTorch 2.5

Python 3.10

GPU: CUDA12.6

환경 이름 (수동) (선택)

환경 변수 (선택)

+ 환경 변수 추가

선택 항목

실행 환경

- PyTorch (NGC) - NVIDIA가 배포하는 딥러닝 환경
- 버전 - 24.09 / PyTorch 2.5 / Python 3.10 / CUDA 12.6
- 실습에 필요한 라이브러리가 대부분 포함

나머지 항목

- 환경 이름 / 환경 변수 - 선택 사항, 입력하지 않음

AutoML — 환경 설정

실습 환경 — 실행 환경 & 자원 할당

자원 할당

자원 그룹

default

자원 프리셋

사용자 설정 자원 할당

CPU ?

18

Core

1

60

메모리 ?

MEM

48

GiB

0.625g

240g

Application MEM 47g

공유 메모리 (SHMEM) 1g 자동 ?

AI 가속기 ?

0.3

fGPU

0

1

CPU – **18** Core

메모리 – **48** GiB

공유 메모리 (SHMEM) – **자동**

AI 가속기 – **0.3** fGPU

AutoML — 환경 설정

실습 환경 — 데이터 & 폴더

스토리지

데이터

워크로드

세션

나의 실행 환경

메트릭

자원 요약

통계

폴더 생성 및 파일 업로드

폴더 생성

내 폴더

1 / 30

프로젝트 폴더

0

초대된 폴더

0

프로젝트 Edu

0%

사용자 장승우

0.22%

2.18 GB / 1 TB

폴더

C

폴더 생성

활성 1

휴지통

전체

일반

파이프라인

자동 마운트

모델

이름 ▾

검색

Q

☐	이름	제어	상태	위치	종류	마운트 권한	소유자
☐	SKKU		READY	sp-bai-sc-g010:sas	사용자	읽기 및 쓰기 R W	

전체 1개 중 1 - 1 < 1 > 10 / 페이지

Applied AI & Computer Vision Lab · Sungkyunkwan University

11

AutoML — 환경 설정

실습 환경 — 데이터 & 폴더

새 폴더 추가

사용 방식 ☒ 일반 ☐ 모델 ☐ 자동 마운트 ?

폴더 이름

위치

종류 ☒ 사용자

권한 ☒ 읽기 및 쓰기 ☐ 읽기 전용

초기화

취소

생성

데이터 & 폴더

이름으로 검색

C

+

☐ 이름 (경로 & 대체이름 ?)	사용 방식	호스트	종류	권한	생성됨
☐ SKKU	general	sp-bai-sc-g010:sas	사용자	R W	2026.07.0

☒ 이름 (경로 & 대체이름 ?)	사용 방식	호스트	종류	권한
☒ SKKU				
☒ 경로 & 대체이름	general	sp-bai-sc-g010:sas	사용자	R W
/home/work/SKKU				

AutoML — 환경 설정

실습 환경 — 검토 및 시작

새 세션 시작

최근 기록

세션 타입

수정

세션 타입: interactive

실행 환경

수정

프로젝트: Edu

이미지: Ngc Pytorch | 24.09 | x86_64 | PyTorch 2.5 | Python 3.10 | GPU: CUDA12.6

자원 할당

수정

자원 그룹: default

컨테이너당 리소스 할당: 18 Core 48 GiB (SHM: 1.00GiB) 0.30 fGPU

에이전트: auto 컨테이너 개수: 1

클러스터 모드 설정: 단일 노드

총 자원 할당량

18 Core 48 GiB (SHM: 1.00GiB) 0.30 fGPU

- 세션 타입: interactive
- 행 환경: NGC PyTorch 24.09 (PyTorch 2.5 · Python 3.10 · CUDA 12.6)
- 자원 할당: 18 Core · 48 GiB(SHM 1 GiB) · 0.30 fGPU, 단일 노드 / 컨테이너 1개
- 세 항목 일치 여부 검토 후 세션 시작

AutoML — 환경 설정

실습 환경

대시보드

스토리지

데이터

워크로드

세션

나의 실행 환경

메트릭

자원 요약

통계

세션을 만들어 보세요

세션 시작

나의 총자원 제한

CPU
18 Core

RAM
48 GiB

GPU
0.3 fGPU

사용 중

C

⚙

세션

C

세션 시작

전체 1

Interactive 1

Batch

Inference

업로드 세션

실행중

종료됨

세션 이름

검색

Q

세션 이름	상태	AI 가속기	CPU	메모리	경과 시간	실행 노드
150baIMX-session	RUNNING	0.30 fGPU	18	48 GiB	00:02:31	i-sc-g010

2

클릭

1

전체 1개 중 1 - 1

<

1

>

10 / 페이지

⚙

AutoML — 환경 설정

실습 환경

× 세션 정보

150baIMX-session

세션 ID	7363a1bb-1380-438c-8e13-3b70168adcc6		
상태	RUNNING		
실행 환경	Ngc Pytorch 24.09 x86_64 PyTorch 2.5 Python 3.10 GPU: CUDA12.6		
자원 할당	default 18 Core 48 GiB 0.30 fGPU		
예약 시간	2026년 7월 27일 오전 9:57 경과 시간: 00:07:52		
자원 사용량	CPU 0.1% / 1800% RAM 0.14 GiB / 48 GiB GPU (util) 0% GPU (mem) 0 GiB / 24 GiB I/O Read: 0 MB / Write: 0.26 MB		

커널

호스트명	상태	에이전트 ID	커널 ID	컨테이너 ID
main1	RUNNING	i-sc-g010	dd0963f2-c18c-42a8-8449-82af75c0c761	3beef358d6b918fa054249d73

전체 1개 중 1 - 1 < 1 > 10 / 페이지

1 클릭



앱: 150baIMX-session

Utilities



Console

Development



Jupyter Notebook



JupyterLab



Visual Studio Code

Machine Learning Tools



TensorBoard

사전 개방 포트



mlflow-ui



nniboard

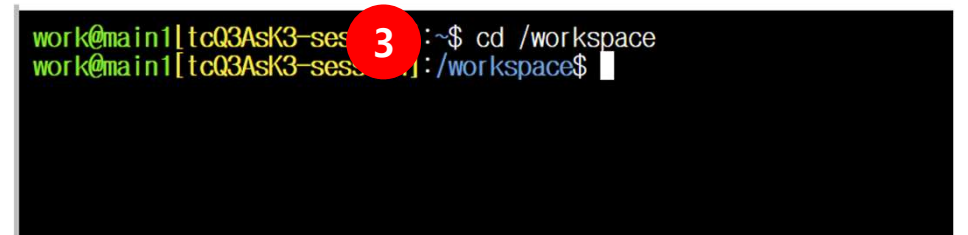
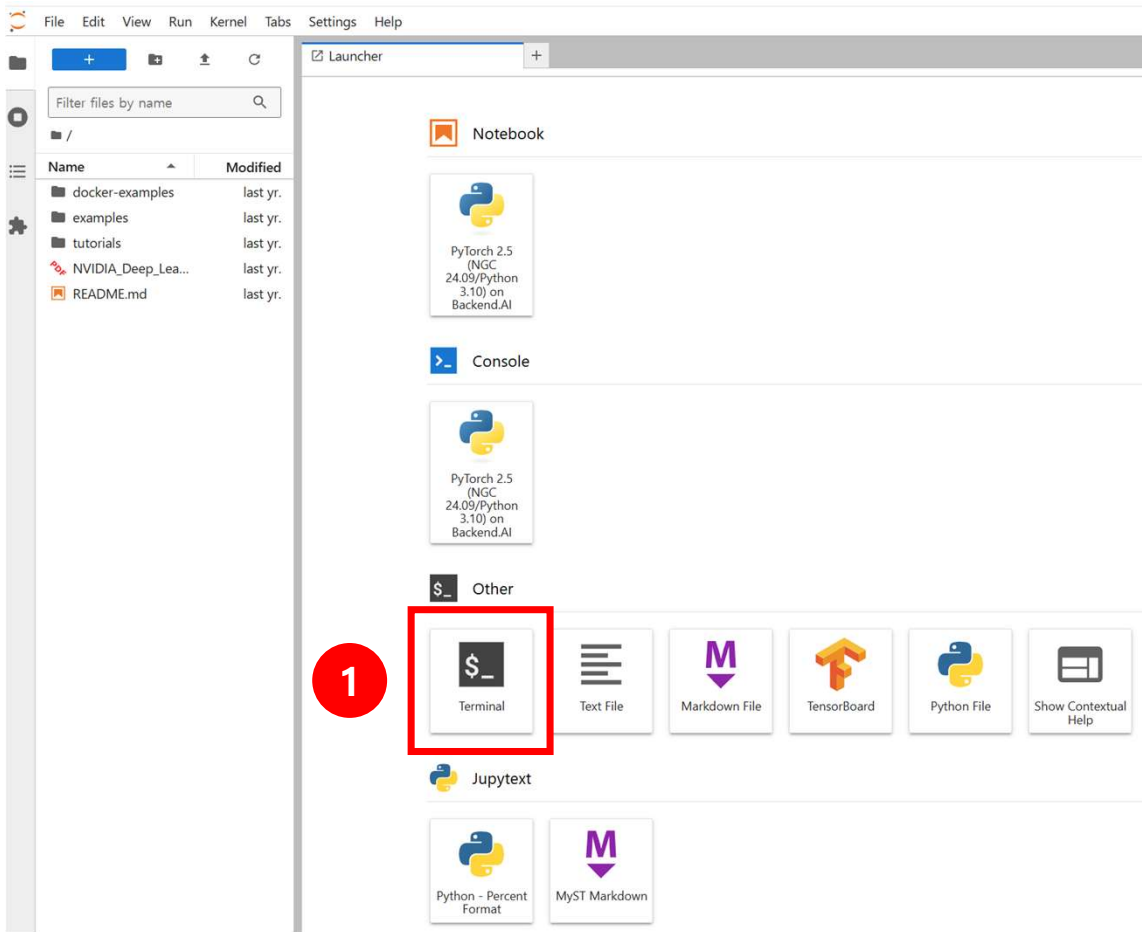
☐ 앱을 외부에 공개

허용된 클라이언트 IP (선택으로 구분)

2 클릭

AutoML — 환경 설정

실습 환경



cd /workspace

AutoML — 환경 설정

실습 환경

```
work@main1[tcQ3Ask3-session]:~$ git clone https://github.com/SE93W0/SKKU_SUMMER_LECTURE.git
Cloning into 'SKKU_SUMMER_LECTURE'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

Name	Modified
/	
docker-examples	last year
examples	last year
SKKU	2 days ago
SKKU_SUMMER_LECTURE	24 seconds ago
tutorials	last year
NVIDIA_Deep_Learning_Co...	last year
README.md	last year

Name	Modified
Day1	3h ago
Day2	7h ago

Name	Modified
tabular	4h ago
TimeSeries	3h ago
install.ipynb	3h ago
SeoulBikeData.csv	20d ago

```
[*]: !pip install -U pip
!pip install -U setuptools wheel
!pip install fg-data-profiling
!pip install -U ipywidgets
!pip install msgpack
!pip install autogluon
!pip install -U accelerate transformers
!pip install -U bitsandbytes
```

클릭 -> Shift + Enter

AutoML

개요

머신러닝 활용의 확산 — 전문성은 그대로, 장벽만 낮춘다

AutoML은 전문가를 대체하지 않고, 반복 작업을 대신 수행

AutoML이 하는 일

- 코드·환경 설정 (코드 몇 줄)
- 일반적인 전처리·검증 반복 작업
- 여러 모델 학습·비교·튜닝
- 성능 리더보드 제공 및 최적 모델 저장

연구자가 하는 일

- 데이터 수집 및 품질 확인
- 예측할 대상 설정
- 의미 있는 변수 판단
- 결과 해석·활용

AutoGluon — 이번 실습에서 사용할 도구

AutoML은 개념, AutoGluon은 이를 구현한 오픈소스 라이브러리

무엇인가

- AWS AI 개발 오픈소스
- 2019년 공개
- 정형·시계열·이미지 지원
- 무료·상업적 활용 가능

왜 이 도구인가

- 코드 세 줄로 학습
- 다수 모델 자동 비교
- 앙상블 자동 구성
- 변수 중요도 산출

핵심 함수 네 개

- `fit()` — 학습
- `leaderboard()` — 비교
- `predict()` — 예측
- `feature_importance()` — 해석

AutoML — 개요

실습 로드맵



AutoML

정형(Tabular) 데이터

정형 데이터란?

정형 데이터 — 행과 열로 정리된 표 형태의 데이터

열 = 변수(특성)

↓

	기온(°C)	습도(%)	강수량(mm)	시간	계절	공휴일	대여량(대)
	20	55	0	8	봄	아니오	1,120
↖	28	70	5	14	여름	아니오	430
행 = 한 사례	-5	40	0	18	겨울	아니오	560
	15	60	0	3	가을	예	90

↑
정답 열 = 예측 대상

정형 데이터로 무엇을 할 수 있나

표에서 '정답 열'을 정하면, 나머지 열로 그 답을 예측

분류 (Classification)

“어떤 종류인가?”

- 정답이 범주일 때
- 예: 합격/불합격, 정상/불량, 우수/보통/미흡

회귀 (Regression)

“값이 얼마인가?”

- 정답이 숫자일 때
- 예: 주택 가격, 수확량, 미세먼지 농도

변수 중요도 (Feature Importance)

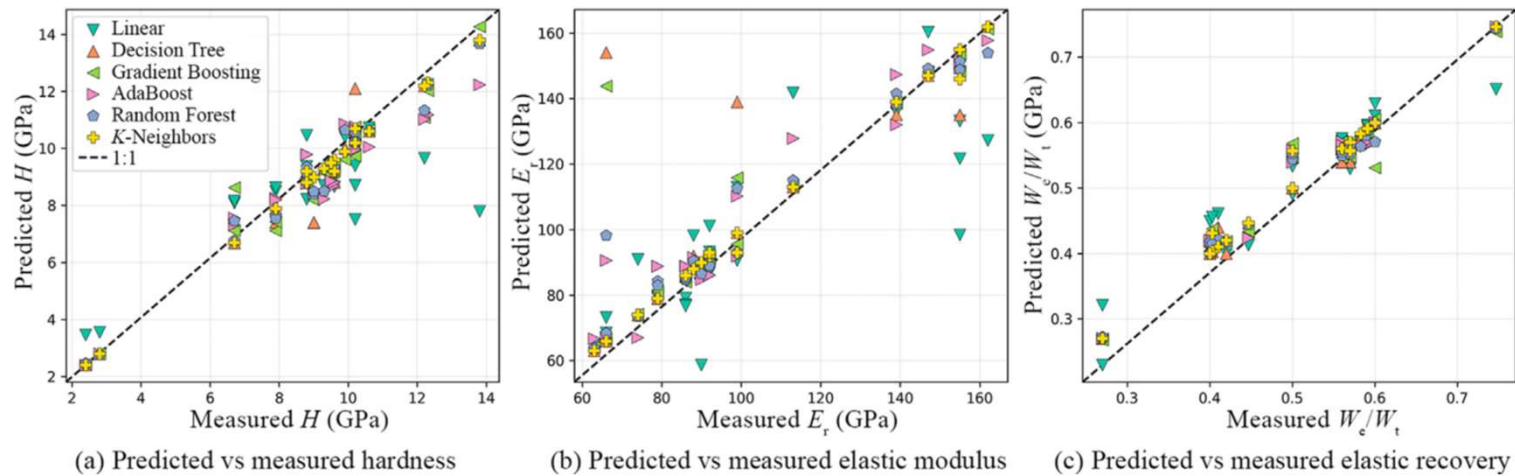
“무엇이 영향을 주는가?”

- 어떤 변수가 결과를 좌우하는지 파악
- 예: 성적에 영향을 준 요인, 불량을 유발한 공정 변수

AutoML — 정형 데이터

달 운석 광물 물성 예측

희귀한 달 운석은 손상 최소화가 필수 — 원소 조성은 비파괴로 측정 가능하나, 물성은 직접 측정이 까다로움
→ 비파괴로 얻는 원소 조성만으로 물성까지 예측할 수 있을까?



세 가지 물성(경도·탄성계수·탄성회복) 모두 예측이 실측선에 밀집

데이터

현미경으로 측정 원소 비율 표 (시료 1개 = 1줄)

시료A → 산소45%·마그네슘12%·규소20%·철8%

모델

분류 = TabPFN (트랜스포머)

물성 예측 = K-최근접이웃(K-NN) 등 6종 비교

결과

광물·광물군 분류 100% (운석 분류 92.3%)

물성 예측 $R^2 \approx 0.99$ (1에 가까울수록 정확)

Machine learning applications on lunar meteorite minerals: From classification to mechanical properties prediction, Peña-Asensio et al., International Journal of Mining Science and Technology (2024)

합금 설계

금속 배합 조합이 사실상 무한 — 실험만으로 최적 조성 탐색은 비현실적
→ 데이터로 최적 조성을 찾고, 강한 이유까지 설명할 수 있을까?

역설계는 어떻게?

조성→물성은 계산되지만, 그 반대(물성→조성)는 어려움 → 후보를 넣어보며 찾아야 함

- ① **목표:** 원하는 물성을 기준으로 설정 (예: 더 단단하게)
- ② **빠른 예측:** 물성 계산은 시뮬레이션이라 느림 → AI가 대신 즉시 예측(대리 모델) → 수만 번 시도 가능
- ③ **반복 개선:** 더 나은 조성을 제안·평가·수정하며 목표에 수렴

데이터

합금 1개 = 한 줄(원소 5종 비율)

정답 열 = 물성 · 시뮬레이션으로 수만 건 생성

모델

물성 예측 + 최적 조합 거꾸로 탐색 + 이유 설명

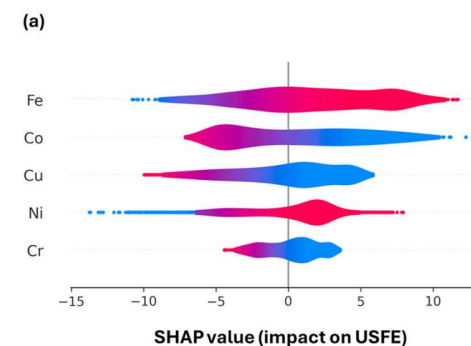
모델: 앙상블 ML·1D CNN

결과

표준(등물) 합금보다 우수한 새 조합 발굴

→ 실제로 합성·검증 (FCC, 예측과 일치)

원소별 영향력 (SHAP) — 위로 갈수록 큼

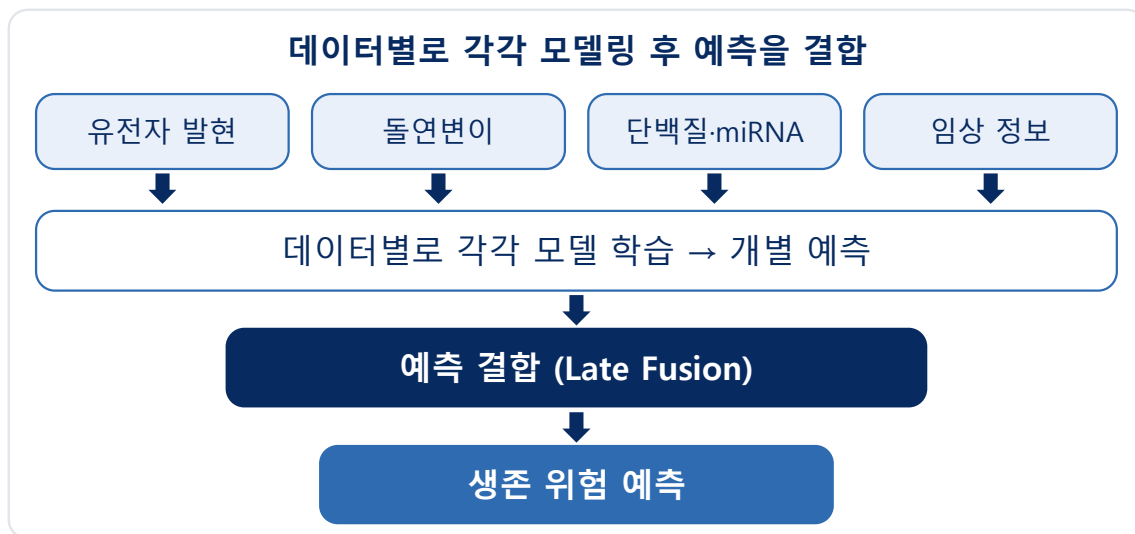


Fe·Co가 물성을 가장 크게 좌우, Cr은 미미

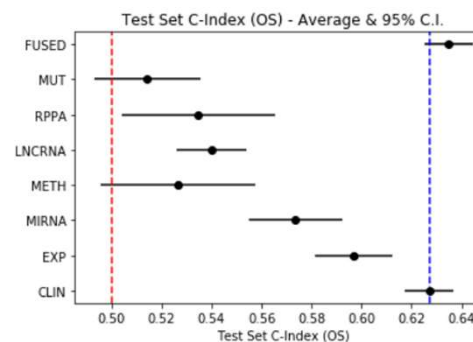
Experimentally validated inverse design of FeNiCrCoCu MPEAs and unlocking key insights with explainable AI, Wang et al., npj Computational Materials (2025)

AutoML — 정형 데이터 암환자 생존 예측

검사 종류는 많지만 환자 수는 적음 — 데이터를 그냥 합치면 과적합으로 오히려 부정확
→ 적은 데이터로도 여러 검사를 잘 활용해 정확히 예측할 수 있을까?



융합 모델(FUSED)이 가장 정확



FUSED가 맨 위·가장 오른쪽 = 단일 데이터보다 정확

데이터

환자 1명당 여러 검사를 모은 표
유전자·돌연변이·단백질·임상 (4~7종)

모델

검사 종류별로 따로 모델 학습
예측 결합 — Late Fusion(앙상블)

결과

33개 암종 중 24개, 단일 대비 우수
범암종 C-index 0.785 (0.5=찍기, 1=완벽)

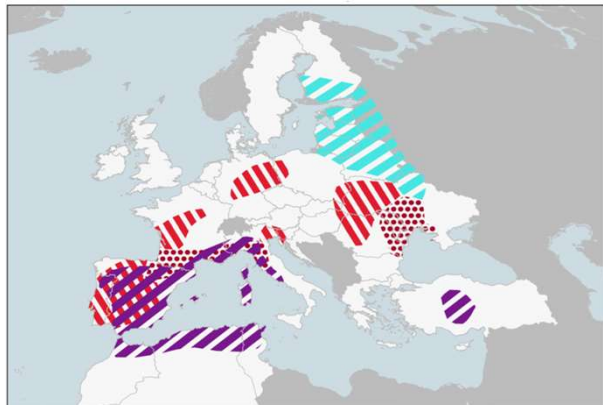
A machine learning approach for multimodal data fusion for survival prediction in cancer patients, Nikolaou et al., npj Precision Oncology (2025)

AutoML — 정형 데이터

기후 재해 탐지

기후 데이터는 폭증하지만 전문가 수작업 탐지는 느리고 주관적
→ AI로 빠르고 일관되게, 근거까지 제시하며 탐지할 수 있을까?

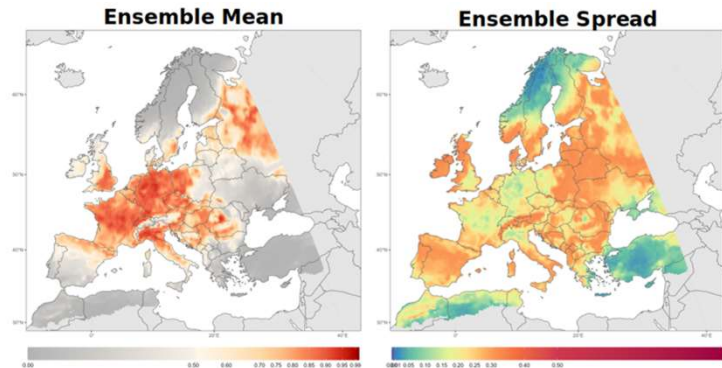
BEFORE — 전문가가 손으로 그린 위험 지도



Cold spell
Drought
Rain deficit
Heat wave



AFTER — 모델이 자동 생성 (위험 + 불확실성)



지역별 재해 위험 확률 + 불확실성 (폭염·가뭄 등 다중 재해)

데이터

지역·시점별 날씨 측정값 표

1줄 = 특정 지역·시점 (기온·강수 등)

모델

전문가 정답 지도로 학습 (XGBoost 앙상블)

다중 재해를 확률로 판정

결과

다중 재해 확률지도 + 불확실성 제공

SHAP로 판단 근거(주요 변수) 제시

Expert-driven explainable AI models can detect multiple climate hazards relevant for agriculture, Essenfelder et al., Communications Earth & Environment (2025)

AutoML

정형 데이터 전처리

데이터 소개: 서울 공공자전거 대여 기록

서울시 공공자전거의 시간대별 대여량 + 그 시각의 날씨 기록

기간: 2017.12.01 ~ 2018.11.30 (1년) · 8,760행 (365일×24시간) · 14개 컬럼

날씨와 시간 정보로 시간당 대여량을 예측 → 회귀(regression) 문제

Date	대여량	Hour	기온(°C)	습도(%)	강수(mm)	Seasons	Holiday
01/12/2017	254	0	-5.2	37	0.0	Winter	No Holiday
01/12/2017	204	1	-5.5	38	0.0	Winter	No Holiday
01/12/2017	173	2	-6.0	39	0.0	Winter	No Holiday

전처리는 왜 필요한가?

형식 · 의미 · 정보 — 세 가지 층위

1. 모델이 읽을 수 있는가?

모델의 입력은 숫자 — 글자·날짜는 그대로 읽지 못함

Seasons = Winter ✗ 글자
Date = 2017-12-01 ✗ 날짜
Temperature = -5.2 ✓ 숫자
그대로는 학습에 쓸 수 없음

2. 그 값을 믿을 수 있는가?

예시 — 기온 열의 999

형식 숫자임 ✓ → numerical로 통과
의미 999°C는 불가능 — '측정 안 됨'의 기록
결과 모델은 999를 진짜 기온으로 학습

- 형식이 맞아도 내용은 틀릴 수 있음
- 모델은 "이 숫자가 말이 되는가"를 판단 못 함

3. 정보를 다 꺼냈는가?

원시 값에 묻힌 정보를 특징으로 추출

자동 날짜 → 연 · 월 · 일 · 요일
사람 출퇴근 시간대 · 체감온도(기온 + 풍속)

AutoML — 전처리

전처리가 하는 일

못 읽는 열 → 읽을 수 있는 열

BEFORE — 원본 · 3열

Seasons	Date	Temp(°C)
Winter	2017-12-01	-5.2
Winter	2017-12-02	-6.0



AFTER — 전처리 후 · 7열 `fit_transform`

Seasons	Date	year	month	day	dayofweek	Temp(°C)
3	1512086400	2017	12	1	4	-5.2
3	1512172800	2017	12	2	5	-6.0

Seasons

Winter → 3 글자는 못 읽음 · 알파벳순 번호

Autumn 0 · Spring 1 · Summer 2 · Winter 3

Date

1열 → **5열** 날짜는 365개가 전부 다른 값 — 반복이 없어 규칙을 못 찾음

요일 7개 · 월 12개로 바뀌야 패턴이 보임

Temperature

그대로 이미 숫자

AutoGluon이 타입을 판별하는 법

값의 패턴 → 타입 자동 판별

판별 결과

판별된 타입	개수	해당 열
('int', ['bool'])	2	Holiday · Functioning Day
('category', [])	1	Seasons
('int', ['datetime_as_int'])	5	Date · Date.year · Date.month · Date.day · Date.dayofweek
('int', [])	3	Hour · Humidity · Visibility
('float', [])	6	Temperature · Wind speed · Dew point · Solar Radiation · Rainfall · Snowfall

판별 기준

타입	기준	처리
bool	고유값이 딱 2개	0 / 1 (int8)
category	문자열 + 값이 반복됨	정수 코드 부여 (알파벳순)
datetime	날짜 변환에 성공	연·월·일·요일 분해 + 원본은 타임스탬프
int / float	이미 숫자	그대로 통과
text	문자열 + 대부분 고유 + 여러 단어	단어 빈도 · 글자수

쓸모없는 열은 자동 삭제

정보가 없는 열 → 알아서 제거

어떤 열이 삭제되는가

학번	나이	국가	나이_사본
20231001	34	한국	34
20231002	51	한국	51
20231003	27	한국	27

유형	예시	왜 쓸모없나
상수 열	국가 — 전부 "한국"	값이 하나뿐 → 행을 구분할 수 없음
중복 열	나이_사본 = 나이	같은 정보가 두 번 — 새로 알려주는 게 없음
식별자 열	학번 — 전부 다른 값	8,760명이면 8,760개 값 — 외울 수는 있어도 일반화 불가

왜 삭제해도 안전한가

- 상수 열 — 어떤 기준으로 나뉘도 전부 한쪽으로 감
- 중복 열 — 남은 한 열이 같은 정보를 그대로 담고 있음

왜 삭제해야 하는가

- 학습 속도·메모리 낭비
- 중요도 해석이 왜곡됨 — 같은 정보가 두 열로 갈리면 중요도가 절반으로 나뉨

결측(NaN)은 채우지 않는다

빈칸 → 그대로 모델에

결측이 있는 데이터

날짜	시각	기온(°C)	습도(%)	대여량
05-21	2	13.9	45	262
05-21	3	13.0	(빈칸)	165
05-21	4	12.4	(빈칸)	113
05-21	5	11.9	52	200



채운다 — 일반적 방식

습도 = 58.2 (전체 평균)

새벽 3시 습도가 연평균이었을 리 없음

빈칸이었다는 흔적이 사라짐

채우지 않는다 — AutoGluon

습도 = NaN

비어 있다는 사실이 정보로 남음

모델이 각자 알아서 처리

LightGBM · XGBoost

CatBoost

신경망

빈칸을 어느 쪽으로 보낼지 데이터를 보고 결정

빈칸을 한쪽으로 몰아 따로 다룸

내부에서 자체적으로 값을 채움

서로 다르게 처리 → 앙상블 다양성 ↑ = 성능 ↑

AutoML — 전처리

코드로 확인하기

준비 → 변환 → 확인

```
import pandas as pd
from autogluon.features.generators import AutoMLPipelineFeatureGenerator

df = pd.read_csv("SeoulBikeData.csv", encoding="latin-1")
df["Date"] = pd.to_datetime(df["Date"], format="%d/%m/%Y")

X = df.drop(columns=["Rented Bike Count"])    # 예측 대상 제외

gen = AutoMLPipelineFeatureGenerator()
out = gen.fit_transform(X)    # ← 전처리 전체가 이 한 줄
```

변환 결과 확인

```
print(X.shape, "→", out.shape)    # (8760, 13) → (8760, 17)
print(out.columns.tolist())
```

`X = df.drop(...)` 예측 대상(label)은 전처리 대상이 아님 — 먼저 분리
`fit_transform` 타입 판별 → 변환을 한 번에 수행

값을 믿을 수 있는가?

형식 해결 → 내용 확인

- 형식 문제는 코드 한 줄로 해결 — 글자 없음 · 빈칸 없음 · 13열 → 17열
- 남은 질문 — ② 값을 믿을 수 있는가 · ③ 정보를 다 꺼냈는가
- **답하려면 데이터를 직접 봐야 함**

탐색적 데이터 분석 (EDA) — 모델 적용 전, 데이터의 구조와 품질을 파악하는 단계

확인 순서		EDA로 얻는 것	
① 규모와 구조	행·열 수, 각 열의 타입	문제 파악	어떤 전처리가 필요한지 판단
② 분포	값이 몰린 구간, 치우침 여부	가설 확인	예상한 관계가 실제로 있는지
③ 결측과 이상치	빈 값, 범위를 벗어난 값	예측 가능성 판단	데이터에 신호가 있는지
④ 중복	중복 행, 다른 열과 동일한 열	해석의 근거	나중에 모델 결과를 검증할 기준
⑤ 변수 간 관계	예측 대상과 함께 움직이는 변수		

- **도구가 보여주는 것** — ①~④ 전부, ⑤는 선형 관계만
- **사람이 해야 하는 것** — 그 값이 말이 되는지 판단 · 비선형 관계는 직접 확인

자동 EDA 도구 — Data Profiling

표만 넣으면 → 리포트 자동 생성

```
from data_profiling import ProfileReport

report = ProfileReport(df, title="따릉이 EDA")

report.to_notebook_iframe()      # 노트북 안에서 바로 확인
report.to_file("eda.html")      # 파일로 저장
```

섹션	확인 내용	따릉이에서는
Overview	행·열 수, 결측·중복 총량	8,760 × 14 · 중복 0
Alerts	자동 감지된 데이터 품질 문제	높은 상관 · 0 편중 등
Variables	변수별 분포·통계량	평균·최소·최대·고유값 수
Correlations	변수 간 상관관계	기온 0.539 · 기온 ↔ 이슬점 0.913
Missing values	결측 위치·비율	0건
Duplicate rows	중복 행	0건

리포트는 숫자를 보여줄 뿐, 무엇이 문제인지는 말해주지 않는다.

숫자는 관계까지, 모양은 그려야

숫자로만 안 보임 · 그려야 보임

Correlations 섹션

Hour ↔ 대여량 0.425

- 숫자만으로는 — 관계가 있다는 것까지만
- 상관계수 = 직선 관계의 강도
- 곡선·다봉 형태 → 반영 안 됨

그래프가 말하는 것

- 출근 8시 · 퇴근 18시 — 두 번의 봉우리
- 8시 → 10시 절반으로 급감 후 재상승
- 최저 137 · 최고 1,554 — 11배

리포트에 이 그래프 없음

- Variables의 Hour 분포 → 24개 값이 365개씩 완전 균등 · 봉우리 없음
 - Correlations → 0.425 하나뿐
- 숫자는 관계의 존재만 · 모양은 그려야



리포트가 찾아낸 것 — Alerts 12건

자동 감지 → 사람이 해석

유형	내용	판단
높은 상관 예측 대상과	대여량 ↔ 기온 (0.539)	예측에 쓸 신호 단, 너무 높으면 누수 의심
높은 상관 변수끼리	기온 ↔ 이슬점 (0.913) 계절 ↔ 기온	중복 정보 성능보다 해석이 왜곡됨
불균형 2건	Holiday 0.72 · Functioning Day 0.79	실제 현상 — 휴일은 원래 적음
has N zeros	강수 94.0% · 적설 94.9% · 일사 49.1%	비는 원래 안 옴 · 밤엔 일사 0
has N zeros	Hour 4.2% (365건)	0시 = 자정 — 정상
has N zeros	대여량 3.4% (295건)	?

같은 "높은 상관" 경고도 뜻이 다르다 예측 대상과 → 신호 · 변수끼리 → 중복
8번에서 본 "중복 열 자동 삭제"는 완전히 같을 때만 — 0.913은 안 지워짐

대여량 0이 295건 — 왜?

발견 ① — 대여량 0이 295건

형식 정상 · 질문이 잘못됨

① 발견 — Functioning Day와 완전히 일치

- 운영 중단일 대여량 — 예외 없이 전부 0

대여량 = 0	295건
Functioning Day = No	295건
둘 다 해당	295건

② 문제 — 답을 미리 아는 행

Functioning Day = 0 → 대여량 = 0

- 모델이 이 규칙을 배운다 — 우리가 이미 아는 것을
- 예측하려는 것은 "운영 중인 시간의 수요" — 다른 질문이 섞임
- 실제로 쓸 때도 운영 여부는 미리 알고 있음

③ 조치

```
df = df[df["Functioning Day"] == "Yes"].drop(columns=["Functioning Day"])
```

최솟값 0 → 2 1년 내내, 자전거가 한 대도 안 나간 시간은 없었다

0은 수요가 없어서가 아니라 서비스가 없어서

발견 ② — 습도 0%가 17건

형식은 정상 · 의미는 불가능

리포트 화면 Humidity(%) Minimum 0 · Maximum 98 · Missing 0

- `isna()` → 0건 · Alerts에도 없음 (0.19%라 임계값 미달)
- 형식은 완벽한 숫자 → AutoGluon도 그대로 통과

이슬점으로 실제 습도 역산

날짜	시각	기온	이슬점	기록된 습도	계산된 습도
05-22	3	15.5	10.4	0	71.6
05-22	4	15.6	10.5	0	71.7
05-21	3	13.0	-2.5	0	34.0
05-28	7	16.8	8.9	0	59.6

- 정상 행 8,743건 — 평균 오차 0.24%p → 이슬점 열은 신뢰 가능
- 그 기준에서 17건만 어긋남 → 실제 습도 33 ~ 72%

`isna()`는 0건

결측을 0으로 기록하면 결측으로 안 잡힘 · 형식이 숫자면 도구는 통과

결측의 패턴

언제 비었나 → 왜 비었나

- **기간** — 2018.5.19 ~ 5.28 열흘에 집중
- **시각** — 전부 새벽 2~7시

시각	0-1	2	3	4	5	6	7	8-23
건수	0	1	3	4	4	4	5	0

- 무작위 오류 → 24시간에 고른 분포
 - **실제** → 열흘간 새벽에만 반복 · 오후·저녁 0건
- 결론** — 특정 시간대 기기 문제

조치

```
df.loc[df["Humidity(%)"] == 0, "Humidity(%)"] = np.nan
```

- 평균 58.2%로 대체 → 실제 33.1%인 행도 58.2
- **NaN 유지** → 모델이 각자 처리

AutoML — 전처리

③ 정보를 다 꺼냈는가 — 도메인 파생 변수

모델이 못 찾는 조합

만들어본 변수

파생 변수	계산	왜
체감온도	기온 + 풍속	5°C에 바람 불면 실제로는 0°C — 자전거 탈지 결정하는 건 이쪽
불쾌지수	기온 + 습도	30°C라도 습하면 안 나감
평일 출퇴근	요일 + 시각	평일 8시와 일요일 8시는 완전히 다름
비 올 여부	강수 > 0	1mm든 30mm든 "비 오면 안 탄다"

만들었으면 반드시 측정 (AutoGluon · R²)

학습 데이터	기본	파생 추가
전체 6,772행	0.960	0.960
500행	0.865	0.880
200행	0.735	0.776

- 따름이 전체 — 차이 **0.000**
트리가 이미 조합을 찾아내고 있었음
- **데이터가 줄수록 효과가 커짐**
배울 여유가 없으면 미리 만들어줘야 함

만들기 전에는 알 수 없음 도메인 지식으로 만들고, 숫자로 확인한다

AutoML — 전처리

정리 — 어디까지 말기고, 어디부터 내가 보는가

형식은 자동 · 의미는 사람

AutoGluon이 알아서

글자 → 숫자 · 날짜 → 연·월·일·요일

열의 타입 자동 판별

상수 열 · 중복 열 삭제

결측을 채우지 않고 모델별 처리

날짜에서 기본 파생 변수 생성

내가 확인해야

숫자로 위장한 결측 — 습도 0%가 17건

답을 이미 아는 행 — 대여량 0 = 운영 중단 295건

완전히 같지 않은 중복 — 기온 ↔ 이슬점 0.913

범주형 지정 여부 — 재 봐야 앎

도메인 파생 변수 — 체감온도 · 출퇴근

처음의 세 가지 질문

- ① 읽기 가능 — 형식 → AutoGluon
- ② 값 신뢰 — 의미 → 사람
- ③ 정보 추출 — 표현 → 사람

AutoML — 전처리

실습 — 데이터 준비와 학습 전 확인

- 강의에서 본 EDA → 전처리 과정 그대로 실행
- **01_EDA.ipynb** → **02_Preprocessing.ipynb**
- 실습 데이터셋 중 하나 선택

1 · 불러오기

구분자 · 인코딩 · 날짜 열 to_datetime 변환

2 · EDA

ProfileReport(df) → Alerts가 진짜 문제인지 판단

3 · 학습 전 확인

위장 결측 · 누수 변수 · 숫자로 위장한 범주

AutoML — 실습 코드

① 데이터 로드와 프로파일링

- 파일 경로를 보유 데이터에 맞춰 지정한 후, 이하 코드는 원문대로 실행
- 경로·구분자·인코딩 3개 항목 확인 시 프로파일링 이후 단계는 자동 수행

```
# 1) 데이터 로드 ← 여기만 본인 환경에 맞게 수정
import pandas as pd
from ydata_profiling import ProfileReport

df = pd.read_csv("data/train.csv")      # 경로 확인 필수

print(df.shape)
print(df.dtypes)
print(df.isna().mean().sort_values(ascending=False).head(10))

# 2) 프로파일링 리포트 생성 - 이하 수정 없이 실행
profile = ProfileReport(df, title="EDA Report", minimal=True)
profile.to_file("reports/eda.html")
profile.to_notebook_iframe()
```

AutoML — 전처리 실습 데이터셋

Pump it Up — 탄자니아 급수시설 상태 예측 · 3분류 · 59,400행

급수펌프의 설비·위치·관리 정보로 작동 상태를 예측하는 정부·시민 결합 데이터

Absenteeism — 직장 결근 시간 예측 · 회귀 · 740행

브라질 택배회사 직원 36명의 교통비·통근거리·건강 정보로 결근 시간을 예측

Diabetes 130 — 당뇨 환자 재입원 예측 · 분류 · 101,766행

미국 130개 병원의 당뇨 입원 기록으로 30일 내 재입원을 예측하는 전자의무기록 10만 건

Heart Failure — 심부전 환자 사망 예측 · 이진분류 · 299행

심부전 환자 299명의 혈액검사값으로 사망을 예측하는 소규모 의료 데이터

AutoML — 전처리

실습 데이터셋

Myocardial Infarction — 심근경색 합병증 예측 · 분류 · 1,700행

병력·검사·심전도 124개 변수로 합병증을 예측하는 결측·불균형·누수 고난도 데이터

AI4I 2020 — 설비 고장 예측 · 이진분류 · 10,000행

밀링 머신의 온도·회전속도·토크·마모로 고장을 예측하는 예지보전 표준 데이터

Concrete — 콘크리트 압축강도 예측 · 회귀 · 1,030행

시멘트·물·골재 배합비와 양생기간으로 강도를 예측하는 재료공학 데이터

Superconductivity — 초전도체 임계온도 예측 · 회귀 · 21,263행

원소 조성에서 계산한 물성으로 임계온도를 예측해 신소재를 실험 전 선별

AutoML — 전처리

실습 데이터셋

Crop Recommendation — 작물 추천 · 다중분류 · 3,867행

에티오피아 농지의 토양 성분과 계절 기후로 12종 중 적합 작물을 추천

Forest Fires — 산불 피해 면적 예측 · 회귀 · 517행

포르투갈 공원의 기상 조건으로 산불 소실 면적을 예측하는 저신호 난제 사례

* 각 데이터 상세는 *.txt 참조

AutoML

정형 데이터 분할

데이터 분할의 목적

분할이 필요한 이유

- 학습에 쓴 데이터로 측정 → 이미 접한 사례를 다시 평가 → 실제보다 높게 측정됨
- 학습에 쓰지 않은 데이터로 평가해야 일반화 성능 확인 가능

분할 구조



AutoGluon이 자동 분할

→ 연구자가 나누는 것은 학습용 / 평가용 둘

비율 정하기

- 일반적 비율 — 80/20 또는 70/30
- 학습용 ↑ = 더 잘 배움 · 평가용 ↑ = 측정이 안정적
- **기준은 비율보다 평가용의 절대 개수 — 너무 적으면 불안정**

랜덤으로 나누면 안 되는 데이터

값의 문제 아님 · 나누는 방법의 문제

같은 데이터, 같은 모델 — 나누는 방법만 다르게

	R ²	평균 오차
랜덤 분할 (기본값)	0.959	66대
시간순 분할	0.835	163대

오차가 2.5배

66대 → 163대 (전체 평균 729대)

R² — 예측이 실재를 설명한 비율 (1 = 완벽, 0 = 평균만 답하는 수준) · 평균 오차 — 시간당 대여량을 평균 몇 대 틀렸는가

왜 그런가

학습과 평가에 '같은 것'이 들어가서

랜덤 6월 3일 14시·16시로 학습 → 15시를 평가
예측이 아니라 사이 채우기

시간순 학습 2017.12~2018.09 · 평가 2018.09~11
학습에 없던 시기를 예측 → 이것이 현실

묶임의 단위

시간 — 반복 관측

개인 — 같은 학생·환자

집단 — 같은 학교·병원

랜덤 분할하면

미래로 과거를 예측

이미 본 사람을 다시 맞춤

그 집단의 특성을 외움

```
d = d.sort_values(["Date", "Hour"])
n = int(len(d) * 0.8)
train, test = d.iloc[:n], d.iloc[n:]
```

분할의 편향: 소규모 데이터에서의 주의점

문제 상황

예: 제품 200개 중 불량품이 20개인 데이터

- 무작위 분할 시, 우연에 의해 불량품이 한쪽에 집중될 수 있음
- 평가용에 불량품이 2~3개만 포함되면 → 평가 결과가 불안정
- 학습용에서 불량품이 부족하면 → 모델이 불량 패턴을 학습하지 못함

데이터 규모에 따른 차이

대규모 데이터

무작위 분할로도 양쪽 분포가 유사해짐

소규모 데이터

우연에 의한 쓸림 위험이 커짐

해결 방안: 층화 추출 (Stratified Sampling)

주요 그룹의 비율이 학습용과 평가용에서 동일하도록 분할

예: 불량률 10% → 학습용·평가용 모두 10% 유지

※ AutoGluon은 내부 검증 분할에 층화 추출을 자동 적용 (분류 문제)

※ 연구자가 수행하는 평가용 분할은 연구자가 직접 확인해야 함

AutoML — 데이터 분할

한 번의 분할 → K번의 분할

교차검증 (Cross Validation)

단일 분할 — 성능 추정값이 하나뿐

- 300행 → 학습 240 · 평가 60
 - 얻는 것은 측정값 1개
 - 그 값은 어떤 60개가 평가용이 되었는가에 의존
 - random_state 변경 → 다른 60개 → 다른 값
- 이 값이 우연히 높은지 낮은지 알 방법이 없음
- 두 모델을 비교해도 — 차이가 실력인지 우연인지 구분 불가

해결 — 여러 번 나누고 평균

전체를 K조각으로 나눔 (예: K=5)

	조각 1	조각 2	조각 3	조각 4	조각 5
1회	평가	학습	학습	학습	학습
2회	학습	평가	학습	학습	학습
3회	학습	학습	평가	학습	학습
4회	학습	학습	학습	평가	학습
5회	학습	학습	학습	학습	평가

→ K번 측정한 결과의 평균

AutoGluon — num_bag_folds=5

- 수천 행 이상 — 단순 분할로도 충분
- 수백 행 — 교차검증이 사실상 필수

AutoML

정형 데이터 모델 학습

다양한 모델의 학습과 앙상블

최적의 모델은 사전에 알 수 없다

→ AutoGluon은 다양한 모델을 학습한 뒤 성능을 비교하여 선택

트리 계열

LightGBM · XGBoost · CatBoost · RandomForest · ExtraTrees

신경망

NeuralNet

거리 기반

KNN

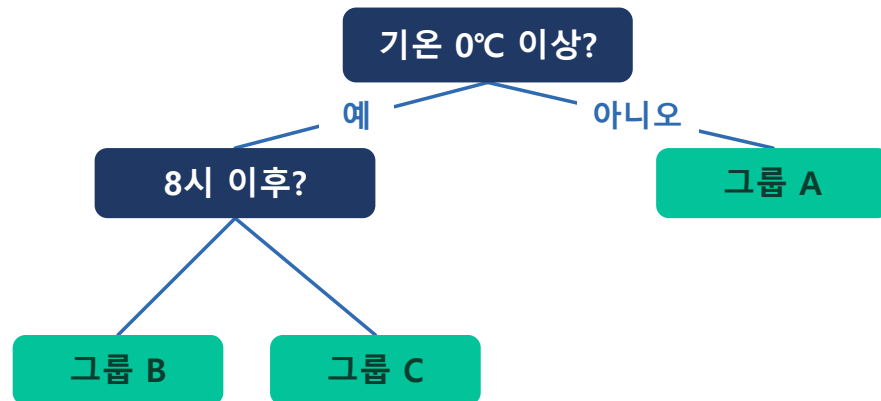
파운데이션

TabPFN · Mitra

모델 — 트리 계열과 신경망

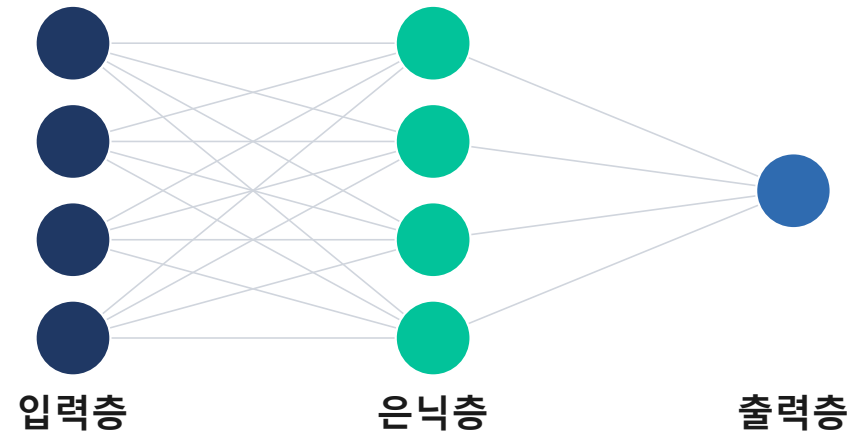
트리 계열

- 데이터를 조건에 따라 반복적으로 분할하여 예측하는 모델
- 각 분기에서 변수 기준으로 데이터를 나누고, 최종 구간의 값으로 예측
- 정형 데이터에서 가장 널리 쓰이는 주력



신경망

- 입력 변수를 여러 층으로 결합하여 복잡한 비선형 관계를 학습하는 모델
- 표 데이터에 맞게 설계된 형태
- 트리 모델과 결정 방식이 달라, 함께 사용 시 상호 보완



모델 — 거리 기반과 파운데이션 모델

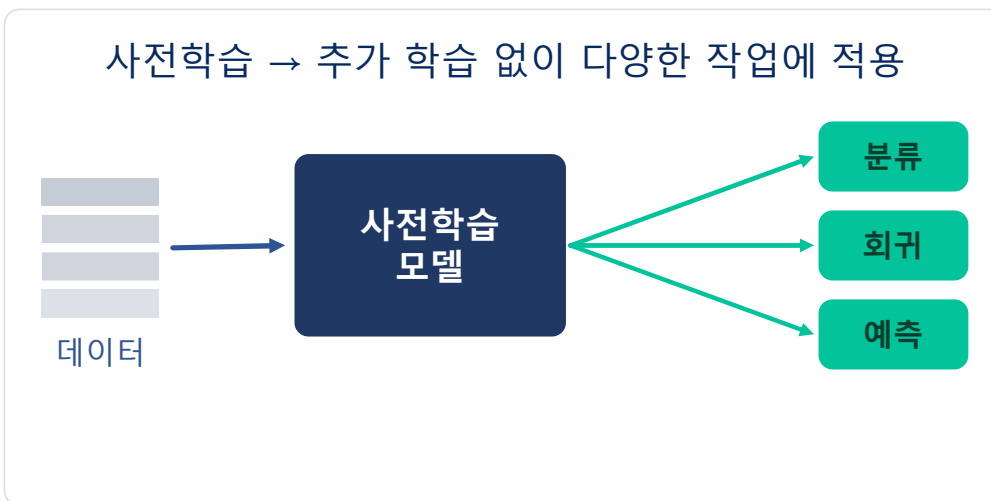
거리 기반

- 가장 가까운 기존 데이터로 새 값을 예측
- 변수 공간의 거리로 유사성을 판단
- 원리가 단순하고 직관적



파운데이션 모델

- 다른 모델: 내 데이터로 처음부터 학습
- 파운데이션: 이미 학습된 모델을 그대로 활용
- 소량의 데이터에서도 작동 - 사전학습 지식 활용



여러 모델을 안정적으로 — 배깅

필요성

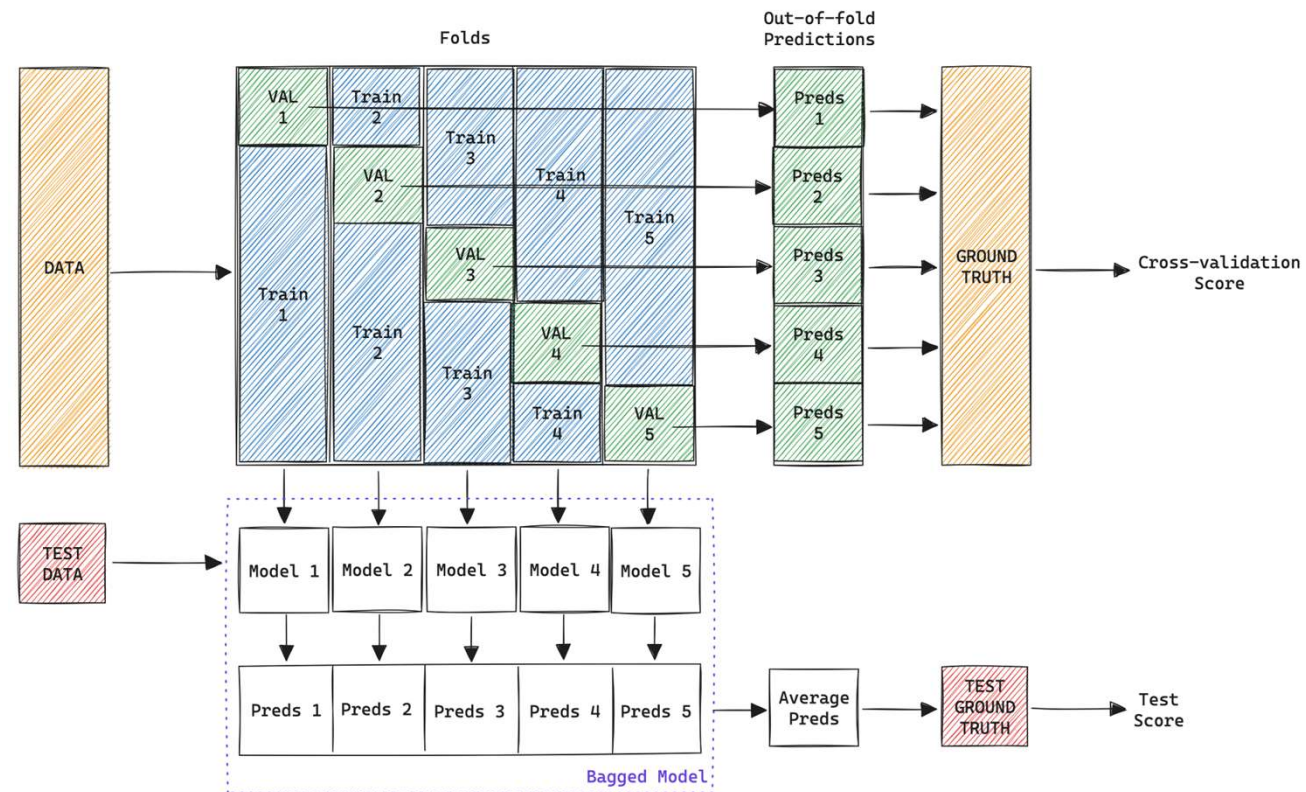
- 같은 모델도 데이터를 나눠 여러 번 학습하면 안정적
- 과적합 ↓ · 예측이 흔들리지 않음

동작 방식 (교차검증과 결합)

- 데이터를 K개 조각으로 나눔
- 각 조각을 뺀 나머지로 학습 → K개 모델 (같은 알고리즘의 복사본 · 학습 데이터만 다름)
- 뺀 조각으로 검증 → 모든 데이터를 활용

결과

- 예측 = K개 복사본의 평균
- OOF 예측 생성 → 스택킹의 재료



AutoML — 모델 학습

모델을 쌓아 올리기 — 스택킹

배깅과의 차이 같은 모델 K개 → 서로 다른 모델 N종

배깅 같은 모델 K개 — 학습 데이터만 다름

스태킹 서로 다른 N종 — LightGBM · XGBoost · 신경망 ...

다층 구조

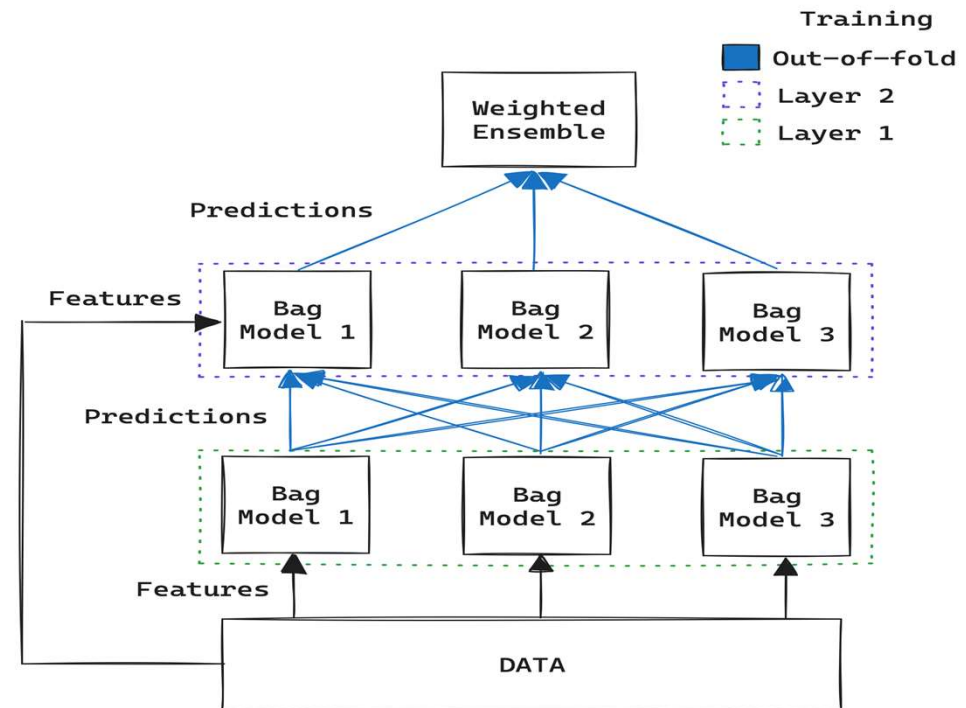
- 각 층 — 여러 종류의 배깅 모델
- 첫 층 — 원본 데이터만
- 다음 층 — 앞 층의 예측 + 원본 데이터

최종 층 — 가중 앙상블

- **WeightedEnsemble** 앞 예측들을 조합
- 각 모델의 의견에 가중치 부여 → 최종 결정
- leaderboard 맨 위에 오는 모델

누수를 막는 장치 — OOF 예측

- 다음 층에 넘기는 것은 OOF 예측
 - 자기가 학습한 데이터의 예측을 넘기면 → 다음 층이 정답을 본 셈
- 각 모델은 학습하지 않은 조각에 대한 예측만 넘김



AutoML — 모델 학습

fit() 한 줄로 학습

나눈 데이터 → 학습 시작

```
from autogluon.tabular import TabularPredictor

predictor = TabularPredictor(
    label="Rented Bike Count",
    eval_metric="r2",          # 성능 판단 기준
).fit(train_data, time_limit=300)
```

로그 — 학습 준비 단계

```
Train Data Rows: 6768 · Columns: 12
Label info (max, min, mean, stddev):
(3556, 2, 687.6, 646.2)
```

```
AutoGluon infers your prediction
problem is: 'regression'
```

```
12 features in original data used to
generate 16 features in processed data
```

```
Automatically generating train/validation
split with holdout_frac=0.1
Train Rows: 6091 · Val Rows: 677
```

로그가 말해주는 것

문제 유형 자동 판별

regression — 대여량이 숫자여서 회귀로 판단

전처리 자동 수행

12개 열 → 16개 특징 (전처리 파트)

검증용 자동 분할

6,768 → 학습 6,091 + 검증 677 (10%)

AutoML — 모델 학습

8개 모델과 앙상블

한 번의 fit() → 9개 결과

Fitting 8 L1 models ...

LightGBMXT		0.9650	12.7s
LightGBM		0.9696	6.1s
RandomForestMSE	..	0.9565	0.7s
CatBoost		0.9733	77.9s
ExtraTreesMSE		0.9550	0.4s
XGBoost		0.9675	8.1s
NeuralNetTorch	...	0.9630	193.1s

← 시간 초과

WeightedEnsemble_L2 0.9752 0.1s

Ensemble Weights:

CatBoost 0.55 · NeuralNetTorch 0.25
XGBoost 0.15 · LightGBM 0.05

Best model: WeightedEnsemble_L2

로그가 말해주는 것

8개 모델을 차례로 학습

각 모델 옆에 검증 점수(r2)

가중 앙상블

개별 최고(0.9733)보다 높은 0.9752

가중치는 균등하지 않음

CatBoost 혼자 절반 이상

최종 선택

검증 점수가 가장 높은 것

성능 확인 — leaderboard()

검증용 점수 → 평가용 점수

```
predictor.leaderboard(test_data)
```

leaderboard 출력

model	score_test	score_val	fit_time
WeightedEnsemble_L2	0.864	0.9752	85.5
LightGBM	0.860	0.970	6.1
CatBoost	0.860	0.973	77.5
XGBoost	0.832	0.967	8.2
LightGBMXT	0.816	0.965	12.6
ExtraTreesMSE	0.812	0.955	0.4
RandomForestMSE	0.788	0.956	0.7
NeuralNetTorch	0.767	0.963	193.6

score_test

평가용 성능 — 학습에 한 번도 쓰지 않은 데이터

score_val

검증용 성능 — AutoGluon이 자동 분할한 677개

fit_time

학습 소요 시간(초)

eval_metric="r2" → 1에 가까울수록 좋음 · 정렬은 score_test 기준

AutoGluon의 점수도 확인이 필요하다

검증 성능과 평가 성능의 비교

모델별 점수 비교

model	검증 성능	평가 성능	차이
WeightedEnsemble_L2	0.975	0.864	-0.111
LightGBM	0.970	0.860	-0.110
CatBoost	0.973	0.860	-0.114
XGBoost	0.967	0.832	-0.136
LightGBMXT	0.965	0.816	-0.149
ExtraTreesMSE	0.955	0.812	-0.143
RandomForestMSE	0.956	0.788	-0.168
NeuralNetTorch	0.963	0.767	-0.196

확인되는 사항

- 예외 없이 부풀려짐 — 8개 모델 전부, 최소 0.110 · 최대 0.196
- 순위가 뒤바뀜 — 검증 6위 신경망(0.963)이 실제로는 꼴찌(0.767)
- 차이의 폭도 제각각 — 어느 모델이 더 부풀려질지 미리 알 수 없음

원인 — 두 가지가 겹침

① 검증 성능이 낙관적

- 검증용 677개가 학습 기간 내부에서 추출
- 같은 시기의 이웃 데이터가 학습에 포함

② 평가 조건이 다름

- 평가 기간 = 2018.09 ~ 11 · 가을만
- 학습에 없던 시기

연구자가 조절하는 학습 파라미터

주요 하이퍼파라미터

옵션	역할	값
presets	성능 ↔ 속도	medium (기본) → best → extreme
time_limit	학습 최대 시간(초)	600 (10분)
eval_metric	성능 판단 기준	회귀 r2 · rmse / 분류 accuracy · f1
num_bag_folds	교차검증 조각 수	5 (분할 5장)

적용 예시

```

predictor = TabularPredictor(
    label="Rented Bike Count",
    eval_metric="r2",
).fit(
    train_data,
    presets="best_quality",  # 정확도 우선
    time_limit=600,         # 10분 제한
)
    
```

presets 세 가지

medium_quality 빠른 학습

best_quality 정확도 최대

extreme_quality 파운데이션 모델 포함 — GPU 필요

표 데이터용 파운데이션 모델

처음부터 학습 → 이미 학습된 것을 활용

일반 모델	내 데이터로 처음부터 학습
파운데이션 모델	대량의 표 데이터로 사전학습 완료 · 그대로 활용

- 추가 학습 없이 다양한 작업에 적용
- 소량 데이터에서 특히 유리 — 사전학습 지식 활용
- AutoGluon에 내장 — **TabPFNv2** · **TabICL** · **Mitra** · **TabDPT** · **TabM**

사용 방법 두 가지

```
# ① preset으로 — 파운데이션 모델 포함 전체 구성
predictor = TabularPredictor(label=LABEL, eval_metric="r2") \
    .fit(train_data, presets="extreme")

# ② 직접 지정 — 파운데이션 모델만
predictor = TabularPredictor(label=LABEL, eval_metric="r2") \
    .fit(train_data, hyperparameters={"TABPFNv2": {}, "MITRA": {}})
```

변수 중요도 측정 원리: 순열 중요도(Permutation Importance)

“이 변수가 없으면 예측이 얼마나 나빠질까?”를 직접 측정

원본 데이터

기온	습도	대여
20	60	340
25	55	480
18	70	210

오차 = 141

‘기온’을 뒤섞음

기온	습도	대여
25	60	340
18	55	480
20	70	210

오차 = 571

성능 하락폭 = $571 - 141 = 430 \rightarrow$ ‘기온’의 중요도

하락폭이 클수록

중요한 변수 — 없으면 예측이 크게 망가짐

하락폭 ≈ 0

예측에 거의 기여하지 않음

음수(-)이면

오히려 해로운 변수 — 제거 시 성능 향상 가능

측정 3단계

1. 정상 예측 — 원본 데이터로 예측해 오차 측정
2. 한 변수 뒤섞기 — ‘기온’ 값만 행끼리 무작위로 섞어 정답과의 관계를 끊고 다시 예측 $\rightarrow$ 오차 측정
3. 하락폭 = 중요도 — $571 - 141 = 430$ 이 곧 ‘기온’의 중요도

AutoML — 모델 학습

변수 중요도 확인 — feature_importance()

변수별 기여도 측정

```
predictor.feature_importance(test_data)
```

중요도 순위

feature	importance	stddev	p_value
Hour	0.998	0.041	3.4e-07
Temperature(°C)	0.269	0.012	4.9e-07
Solar Radiation	0.202	0.006	9.3e-08
Date	0.133	0.011	5.4e-06
Humidity(%)	0.086	0.005	1.5e-06
Rainfall(mm)	0.042	0.003	5.1e-06
Holiday	0.024	0.006	3.5e-04
Dew point temperature(°C)	0.012	0.002	5.7e-05
Visibility · Wind · Snowfall	0.009~0.001		
Seasons	0.000	0.000	0.5

단위 = R² 하락폭 (eval_metric="r2" 기준)

Hour 제거 시 R² 0.864 → 음수로 추락

세 가지 확인 사항

① Hour가 압도적

- 2위의 3.7배 · EDA에서 본 두 번의 봉우리

② Seasons = 0.000

- p_value 0.5 — 아무 기여 없음
- Date.month가 이미 계절 정보를 담고 있음

③ 이슬점 0.012

- 기온과 상관 0.913 (전처리 Alerts)
- 중복 변수 — 기온이 기여를 독점

AutoML — 모델 학습

학습한 모델 사용하기

예측 → 저장 → 재사용

BEFORE — 정답이 없는 새 데이터

Date	Hour	기온	Seasons	대여량
2018-11-30	8	-3.0	Winter	?
2018-11-30	9	-1.5	Winter	?



AFTER — 예측값이 채워짐

Date	Hour	기온	Seasons	대여량
2018-11-30	8	-3.0	Winter	412.6
2018-11-30	9	-1.5	Winter	388.1

`predictor.predict(new_data)`

코드

```
pred = predictor.predict(new_data)          # 새 데이터를 예측

loaded = TabularPredictor.load("ag_ok")      # 새 세션 · 다른 PC
loaded.predict(new_data)                    # fit()이 이미 저장했음
```

입력 열 구성이 학습 때와 같아야 함 — 전처리한 모델 → 전처리한 데이터
새 데이터에도 같은 전처리를 적용 — 함수로 만들어둔 이유

실습 ② — 학습과 해석

따라 하기 · 따름이 강의에서 본 분할 → 학습 → 해석 과정 그대로 실행
03_DataSplit.ipynb → 04_Modeling.ipynb · 실습 ①에서 쓴 데이터로 계속 (또는 다른 데이터셋 선택)

세 단계

4 · 분할

시간·개인·집단으로 묶여 있는지 확인 → 그 단위로 분할

5 · 학습

`TabularPredictor(label=...).fit(train)` · `leaderboard(test)`

6 · 해석

`feature_importance` 로 변수별 기여도 확인

AutoML — 실습 코드

② AutoGluon 학습과 평가

- 별도 split 없이 전체 **df**를 그대로 투입 — 검증셋 분리는 AutoGluon이 내부에서 자동 처리
- **time_limit**과 **presets**만으로 탐색 범위를 조절 — 실습은 짧게 잡고 시작

1) 학습 (내부 검증셋 자동 분리)

```
from autogluon.tabular import TabularPredictor
```

```
predictor = TabularPredictor(label="target", eval_metric="roc_auc").fit(  
    train_data=df,          # 전체 데이터 그대로  
    time_limit=600,         # 초 단위 탐색 예산  
    presets="medium_quality", # best_quality 로 올리면 성능↑ 시간↑  
)
```

2) 모델 비교 - 내부 검증 점수 기준

```
predictor.leaderboard()  
pred = predictor.predict(df)
```

3) 어떤 피처가 기여했는지

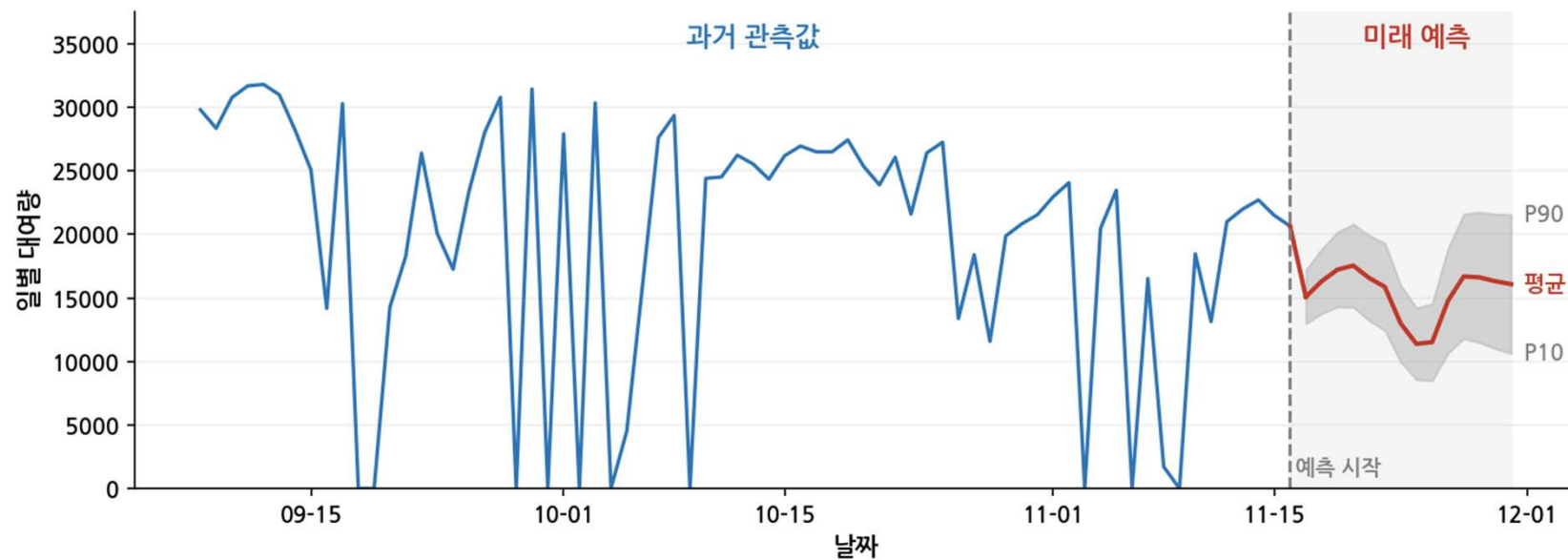
```
predictor.feature_importance(df)
```


AutoML

시계열(Time-series) 데이터

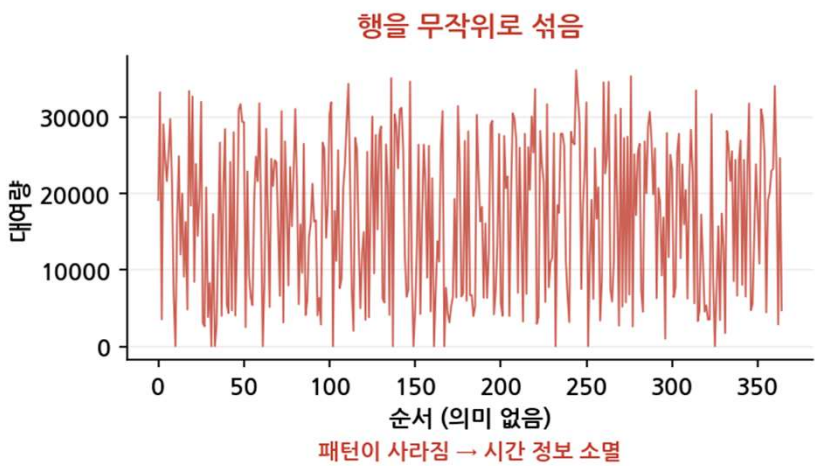
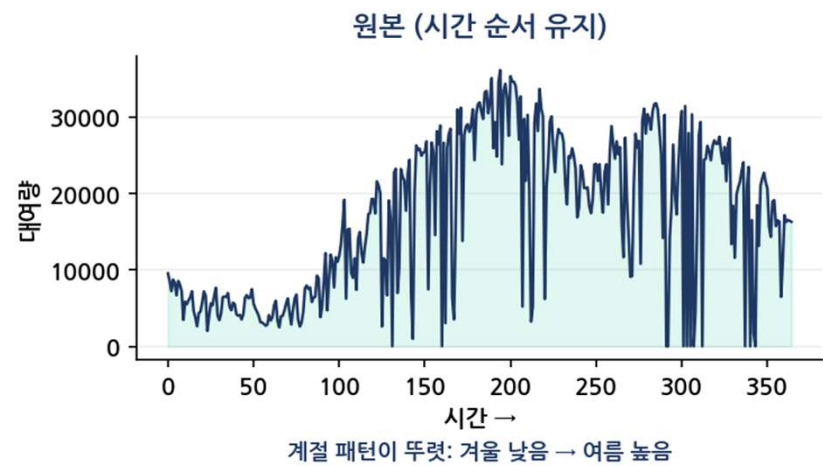
시계열 데이터: 시간 순서를 가진 데이터

시계열 데이터 = 시간 순서에 따라 기록된 데이터



정형 데이터와 시계열 데이터의 차이

핵심 차이는 '순서' — 행을 섞어도 되는가



항목	정형 데이터	시계열 데이터
순서	무관 — 섞어도 됨	핵심 — 섞으면 안 됨
분할	목임이 없으면 무작위 가능 묶여 있으면 그 단위로 (분할 파트)	시간 순 — 과거 학습 · 미래 평가
예측 목표	조건에 따른 값	미래의 값

시계열 데이터로 무엇을 할 수 있나

시간의 흐름 속에서 미래를 예측하고, 패턴을 파악

예측

Forecasting

“이 흐름은 어디로 갈까?”

과거의 추세와 주기를 학습해 다음 시점을 추정

추세·계절성 분석

Decomposition

“어떤 흐름과 주기가 있나?”

장기적 추세와 반복되는 계절성을 파악

추세: 장기적 증감 방향 · 계절성: 주기적 반복

이상 탐지

Anomaly

“평소와 다른 때는 언제인가?”

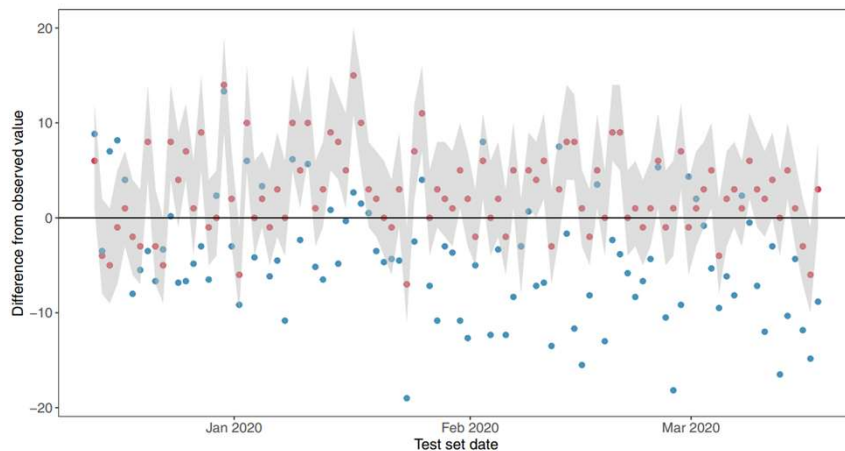
정상 패턴에서 벗어난 지점을 감지

AutoML — 시계열 데이터

응급실 입원 수요 실시간 예측

응급실 과밀은 대기·치료 지연, 환자 안전 위협을 초래
→ 실시간 진료 기록으로 입원 수를 미리 예측해 병상·의료진을 배치할 수 있을까?

실시간 전자의무기록으로 4.8시간 뒤 입원 수를 확률분포로 예측



0.82–0.90

AUROC · 입원 수요 예측 성능

4.0명

평균 오차(MAE) · 기존 관행 6.5명

데이터

- EHR 응급실 방문 21만여 건
- 2019~2021 · 입원율 20.7%
- 시간 순 분할(과거→최근)

모델

- XGBoost로 환자별 입원 확률
- 확률 합산 → 총 입원 수 예측
- RF·로지스틱회귀 대비 최우수

결과

- AUROC 0.82~0.90
- MAE 4.0명 (기존 6.5명)
- 실병원 실시간 시스템 운영

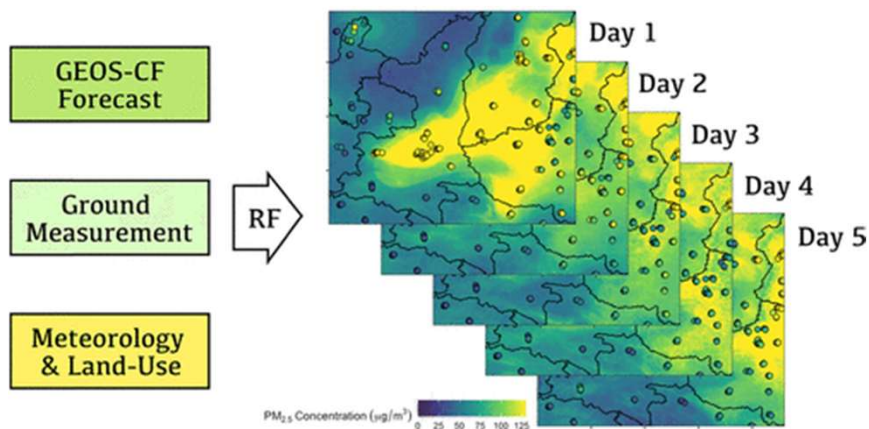
Machine learning for real-time aggregated prediction of hospital admission for emergency patients, King et al., npj Digital Medicine 5, 104 (2022)

고해상도 대기오염(PM2.5) 예측

기존 대기질 예보는 물리·화학 시뮬레이션에 의존 — 정확하나 연산이 무겁고 편향이 큼
→ 머신러닝으로 5일 앞 오염도를 촘촘한 지도로 더 가볍게 예측할 수 있을까?

향후 5일 PM2.5를 1km 해상도로 예측 (중국 중부 펜웨이 평원)

Spatiotemporal Five-Day PM2.5 Concentration Forecast



0.76

1일 뒤 R^2 · 2일 뒤 0.64

1km

고해상도 · 5일 앞 예측

데이터

- 지상 관측 + NASA 공개 위성·예보 데이터
- 특별한 장비 없이, 공개 데이터로 구성

모델

- 랜덤포레스트 앙상블
- GEOS-CF 예보를 학습
- 트리 기반 편향 보정

결과

- 1일 $R^2=0.76$, 2일 0.64
- 5일까지 예측 산출
- 물리모델 편향 보정·경량

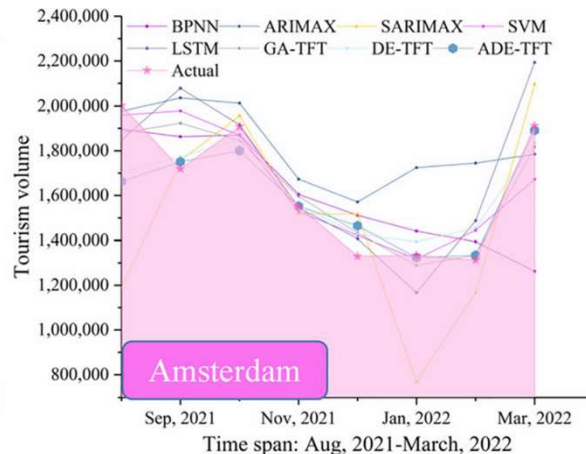
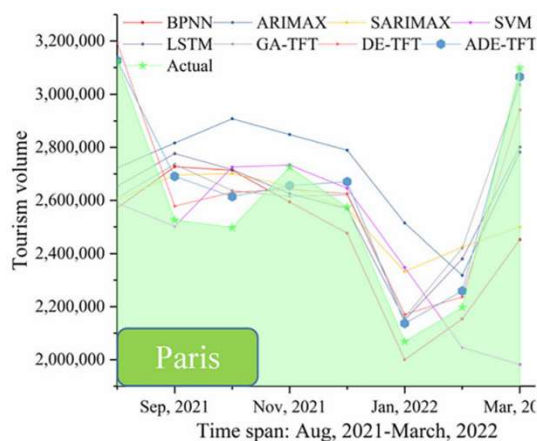
Combining Machine Learning and Numerical Simulation for High-Resolution PM2.5 Concentration Forecast, Bi et al., Environmental Science & Technology 56, 1544–1556 (2022)

AutoML — 시계열 데이터

관광 수요 예측 (코로나19 시기)

코로나19로 관광 수요가 급변 — 과거 데이터만으로는 예측이 어려움
→ 확진자·검색 트렌드를 결합해 급변하는 관광 수요를 예측할 수 있을까?

월별 관광객 수를 예측 · 코로나 급변기에도 실측에 근접



3.1%

예측 오차 (MAPE)

7.4% → 3.1%

여러 데이터 결합 오차 절반 이하로 감소

데이터

- 파리·암스테르담·리스본 월별 관광객 (2012~2022)
- 확진자·구글 트렌드·여행 포럼 결합

모델

- Temporal Fusion Transformer (TFT)
- AutoGluon 딥러닝 · 해석 가능

결과

- ARIMA·SVM·LSTM보다 우수
- MAPE 7.4% → 3.1% (데이터 결합)

Interpretable tourism demand forecasting with temporal fusion transformers amid COVID-19, Wu et al., Applied Intelligence 53, 14493–14514 (2023)

AutoML

시계열(Time-series) 데이터 전처리

시계열 — EDA

시계열 EDA — 무엇을 보는가

정형 EDA는 분포를 본다 · 시계열 EDA는 시간 위의 움직임을 본다

관점	정형 EDA	시계열 EDA
기본 단위	각 변수의 분포	시간에 따른 추이
관계	변수 간 상관	계절성 · 주기
결측	결측 개수 (순서 무관)	결측 시점 · 빈 날짜
숨은 결측	0 · 999로 위장 (전처리 파트)	행 자체가 없음 — <code>isna()</code> 로도 안 잡힘
이상	값의 이상치	급변 구간 (이상 시점)

추이

어디로 가고 있는가

장기 상승·하락, 수준 변화

주기

반복되는 패턴이 있는가

요일·월·연 단위 계절성

이상

언제 튀었는가

급변 구간 · 빈 날짜

시계열 — EDA

시계열 EDA 실습 — tsmode

같은 도구, 두 줄만 다르다

```
from data_profiling import ProfileReport

# Date + Hour → timestamp
df["timestamp"] = df["Date"] \
    + pd.to_timedelta(df["Hour"], unit="h")

profile = ProfileReport(
    df,
    tsmode=True,          ← 시계열 모드
    sortby="timestamp",  ← 시간 기준 정렬
)
profile.to_file("ts_eda.html")
```

시계열 모드에서 추가되는 것

Time-series	시간축 선 그래프 — 전체 흐름
Autocorrelation	반복되는 주기가 있는가
Gap analysis	빠진 시점의 위치와 길이
Statistics	계절성 · 정상성 판정 추가

적용 시점 — 집계 전, 시간별 원본에 적용

일별로 집계하면 하루 안의 리듬이 사라짐 — 따릉이의 핵심 = 시간대별 두 번 피크

→ EDA는 시간별로 먼저 · 예측용 집계는 그 다음

시계열 — 데이터 형식

시계열 데이터의 형식 — 세 개의 열

AutoGluon은 세 개의 열이 필요 — 어느 것 · 언제 · 얼마

item_id	timestamp	target
seoul_bike	2017-12-01	9,539
seoul_bike	2017-12-02	8,523
seoul_bike	2017-12-03	7,222

item_id

어느 시계열인가?

→ 따릉이는 하나, 모두
seoul_bike

timestamp

언제인가?

→ 날짜 (2017-12-01 ...)

target

얼마인가?

→ 대여량 (9,539 ...)

시계열 예측 — 이론 및 실습

집계 방식은 변수마다 다르다

기계는 합계인지 평균인지 모른다

문제 제기 — 시간별 기록 **8,760행**을 일별 **365행**으로 합칠 때
→ 모든 변수를 같은 방식으로 합칠 수 없다

집계 방식	변수	이유
합계 (sum)	대여량 · 강수량 · 적설량 · 일사량	하루 동안 누적된 총량이 의미를 가짐
평균 (mean)	기온 · 습도 · 풍속 · 가시거리 · 이슬점	하루 동안의 대표 상태
첫 값 (first)	계절 · 공휴일 · 운영일	하루 내내 동일한 값

하루 대여량을 평균 내면? → 시간당 평균일 뿐, 그날 몇 대 빌렸는지 알 수 없음
하루 기온을 합계 내면? → 24개를 더한 숫자, 아무 의미 없음

시계열 예측 — 이론 및 실습

여러 변수를 함께 넣기 — 예측을 더 정확하게

기본 3열에 관련 측정값을 열로 더할 수 있다

item_id	timestamp	target	temperature	humidity	...	seasons
seoul_bike	2017-12-01	9,539	-2.45	45.88	...	Winter
seoul_bike	2017-12-02	8,523	1.32	61.96	...	Winter

파란색(기본 3열)은 그대로, 오른쪽에 **관련 변수 10개**를 열로 덧붙임

과거만 아는 값 (8개)

기온 · 습도 · 풍속 · 가시거리 · 이슬점 · 일사량 · 강수량
· 적설량

“지난 날씨는 알지만, 미래 날씨는 아직 모른다”

미래도 아는 값 (2개)

계절 · 공휴일

“다음 주 토요일이 공휴일인 건 달력만 봐도 안다”

실제 사례 — 관련 변수를 더하면?

관광 수요 예측 — 과거 관광량만 사용 시 오차 **7.4%**

+ 확진자 · 검색 데이터 결합 → 오차 **3.1%** (예측 정확도 2배 이상 향상)

시계열 예측 — 이론 및 실습

시계열 데이터 준비 — 자동과 수동

형식만 맞추면, 나머지는 AutoGluon이 자동 처리

AutoGluon이 자동으로

- 시간 간격 파악 — 일별·시간별 자동 판단
- 빠진 시점 채우기 — 날짜 누락 시 행 자동 삽입
- 결측값 처리 — NaN도 모델이 자체 처리

날짜	대여량
3월 1일	9,539
3월 2일	8,523
3월 3일	NaN ← 자동 삽입
3월 4일	7,222

연구자가 할 것

- 시간 간격 지정 — 일별·시간별·월별
- 결측의 의미 판단 — 결측인가 vs "0"인가
→ 수요 예측 시 결측=0이면 0으로 채움

시계열 예측 — 이론 및 실습

따릉이를 시계열 형식으로

원본 14열·시간별 → 시계열 형식 13열·일별

```
# 1) 일별 집계 - 집계 방식은 변수마다 다르게
daily = df.groupby("Date").agg({
    "Rented Bike Count": "sum",      # 합계
    "Temperature(°C)": "mean",      # 평균
    "Seasons": "first",             # 첫 값
    # ... 나머지 변수도 동일하게 지정
}).reset_index()

# 2) 열 이름 정리 + item_id 추가
ts_df = daily.rename(columns={
    "Date": "timestamp",
    "Rented Bike Count": "target",
})
ts_df["item_id"] = "seoul_bike"

# 3) 시계열 형식으로 변환
ts_data = TimeSeriesDataFrame.from_data_frame(
    ts_df,
    id_column="item_id",
    timestamp_column="timestamp",
)
```

► Before — 원본

Date	Hour	Temperature	...	Rented Bike Count
2017-12-01	0	-5.2	...	254
2017-12-01	1	-5.5	...	204

14열 · 8,760행 (365일 × 24시간)



► After — 시계열 형식

item_id	timestamp	target	temperature	...
seoul_bike	2017-12-01	9539	-2.45	...
seoul_bike	2017-12-02	8523	1.32	...

13열 · 365행 · item_id-timestamp = 인덱스

시계열 예측 — 이론 및 실습

빠진 날짜는 자동으로 채워진다

실제 데이터에는 누락된 시점이 존재

```
ts_filled = ts_missing.convert_frequency(freq="D")
```

일 단위(freq="D")만 알려주면 빠진 날짜를 자동으로 채운다

Before — 3~5일 제거 (362행)

timestamp	target	temperature
2017-12-02	8523.0	1.32
2017-12-06	6669.0	1.53

12월 2일 → 곧바로 12월 6일
사이 날짜가 없어 시간이 끊김



After — 3~5일 삽입 (365행)

timestamp	target	temperature
2017-12-02	8523.0	1.32
2017-12-03	NaN	NaN
2017-12-04	NaN	NaN
2017-12-05	NaN	NaN
2017-12-06	6669.0	1.53

빠진 날짜는 행으로 삽입
값은 NaN 그대로

실습 ① — 데이터 준비와 형식 변환

따라 하기 · 따름이

강의에서 본 불러오기 → EDA → 형식 변환 그대로 실행

흐름 익히기

1 · 불러오기

3열 형식(item_id · timestamp · target) 확인

2 · EDA (tsmode)

ProfileReport(df, tsmode=True) 로 추이 · 주기 확인

3 · 형식 변환

TimeSeriesDataFrame · convert_frequency
집계 방식 판단 — 합계? 평균?

AutoGluon TimeSeries — 실습 코드

① 데이터 로드와 EDA

- 파일 경로를 보유 데이터에 맞춰 지정한 후, 이하 코드는 원문대로 실행
- 시계열 모드 프로파일링으로 추세·계절성·결측 구간을 일괄 확인

```
# 1) 데이터 로드 ← 여기만 본인 환경에 맞게 수정
import pandas as pd
from ydata_profiling import ProfileReport

df = pd.read_csv("data/bike.csv", parse_dates=["timestamp"])

df = df.sort_values(["item_id", "timestamp"])
print(df.shape, df["timestamp"].min(), df["timestamp"].max())

# 2) 시계열 모드 프로파일링 - 이하 수정 없이 실행
profile = ProfileReport(df, tsmode=True, sortby="timestamp",
                        title="TS EDA Report")
profile.to_file("reports/ts_eda.html")
profile.to_notebook_iframe()
```

AutoML — 전처리

실습 데이터셋

PJM Energy — 지역별 전력 소비량 예측 · 다중 시계열 · 시간별 · 11개 지역
미국 동부 송전망의 지역별 시간당 소비량을 동시에 예측하는 다중 시계열

Steel Industry — 제철 공장 전력 소비 예측 · 단일 시계열 · 15분 · 1년
전남 광양 제철소의 15분 단위 전력량으로 집계·공변량을 다루는 국내 데이터

Berkeley Earth — 도시별 기온 추세 · 다중 시계열 · 월별 · ~270년
서울 등 세계 100개 도시의 월평균 기온으로 장기 추세와 계절성을 분해

CDC NHSN — 병원 호흡기 입원 예측 · 다중 시계열 · 주별
미국 병원의 주간 독감·코로나 입원 수로 병상 수요를 예측하는 공중보건 데이터

AutoML — 전처리

실습 데이터셋

CDC NWSS — 하수 바이러스 활동 예측 · 다중 시계열 · 주별 · 40개 지점
하수에서 검출된 바이러스량으로 임상 검사보다 먼저 유행을 탐지하는 조기 경보

Melbourne Pedestrian — 도시 보행자 통행량 예측 · 다중 시계열 · 일별 · 10개 센서
멜버른 센서별 보행자 수로, COVID 봉쇄 급락이 담긴 구조 단절 사례

Saugeen River — 하천 유량 예측 · 단일 시계열 · 일별 · 65년
캐나다 소지강 일평균 유량으로 치우친 분포와 장기 계절성을 다루는 최다 정제 데이터

AutoML

시계열(Time-series) 데이터 모델 학습

AutoGluon 시계열이 학습하는 모델들

정형 데이터와 같은 구조 — 시계열용 모델로 교체

통계 모델

ETS · ARIMA · Theta · SeasonalNaive
추세·계절성을 수식으로 포착하는 전통 기법

지역

트리 기반

RecursiveTabular · DirectTabular
시계열을 회귀 문제로 변환 → LightGBM 사용

전역

딥러닝

DeepAR · TemporalFusionTransformer · PatchTST
복잡한 패턴을 신경망으로 학습

전역

파운데이션

Chronos
대량의 시계열로 미리 학습되어 별도 학습 없이 예측

전역

사전학습

시계열 예측 — 모델 학습

지역 모델

지역 모델 (Local)

시계열 하나마다 모델을 따로 학습

예: ARIMA · ETS · Theta · SeasonalNaive

- ✓ 데이터가 적어도 잘 작동
- ✓ 해석이 쉬움 (수식 기반)
- ✗ 유연성이 낮음 (복잡한 패턴 포착 어려움)
- ✗ 예측이 느림 (새 시계열마다 처음부터 학습)



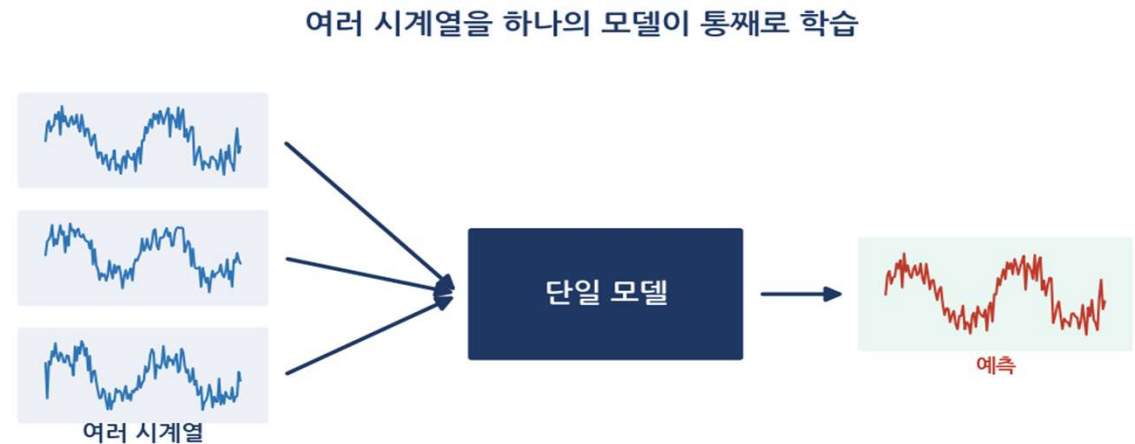
시계열 예측 — 모델 학습

전역 모델

전역 모델 (Global)

여러 시계열을 하나의 모델이 통째로 학습
LightGBM 기반(RecursiveTabular) · DeepAR
· TFT · PatchTST

- ✓ 유연성이 높음 (복잡한 패턴 학습)
- ✓ 예측이 빠름 (학습된 모델 재사용)
- ✗ 학습이 느림
- ✗ 데이터가 많이 필요함



시계열 예측 — 모델 학습

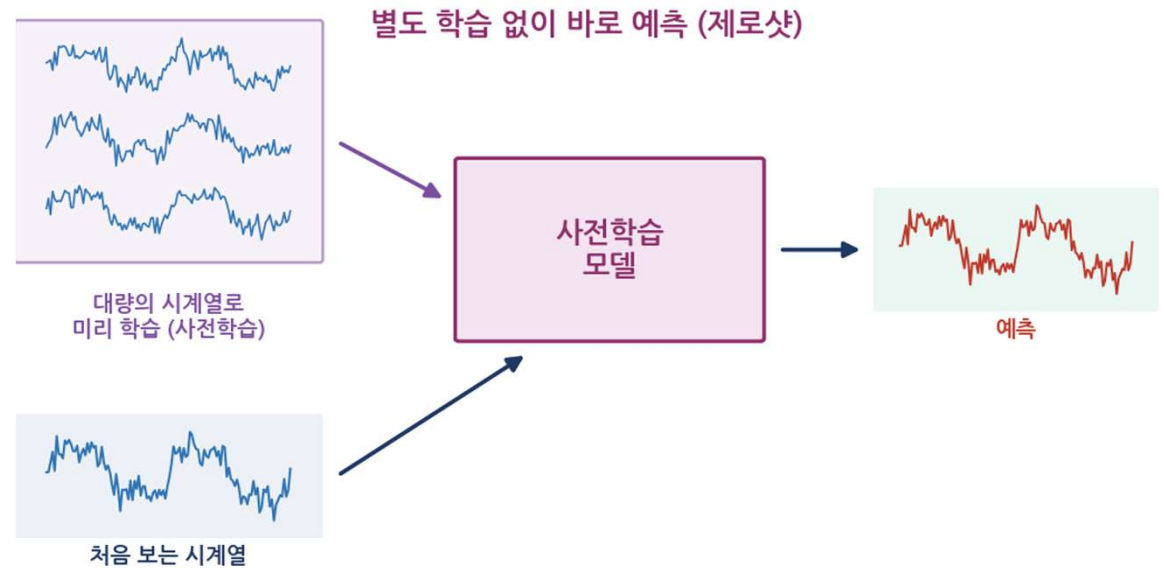
사전학습 모델

사전학습 모델 (Pretrained)

대량의 시계열로 이미 학습되어 있음

예: Chronos

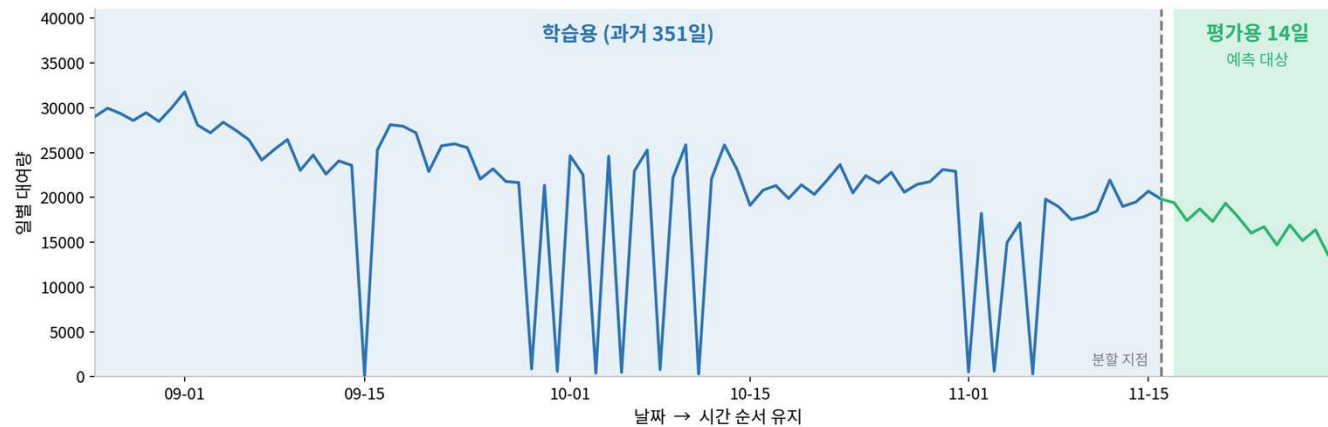
- ✓ 별도 학습 없이 바로 예측 (제로샷)
- ✓ 처음 보는 데이터에도 작동



시계열 예측 — 모델 학습

학습과 평가 분할 — 과거로 배우고, 미래로 시험한다

시계열은 무작위로 나누면 안 된다



왜 무작위로 나누면 안 되나

- 정형: 무작위 80/20 분할 — 행이 독립적, 순서 무관
- 시계열: 무작위로 섞으면 안 됨
 - 미래를 본 채 과거를 맞추는 것
 - 성능이 실제보다 부풀려짐

prediction_length = 14일

```
prediction_length = 14
train_data, test_data = ts_data.train_test_split(
    prediction_length)
```

시계열 예측 — 모델 학습

하나의 시계열로 어떻게 학습하나 — 창을 옮겨가며

정형: 8,760행 = 8,760개 학습 사례 VS 시계열: 365일짜리 딱 하나
→ 학습 사례가 1개뿐인데, 어떻게 학습할까?



창을 옮겨가며 학습 문제를 여러 개 생성
→ 323일에서 수백 개의 학습 사례 확보
각 사례 = "과거를 보고 다음 14일을 맞혀라"

하나의 시계열 → 창을 옮겨가며 수백 개의 학습 사례

시계열 예측 — 모델 학습

시계열 모델 학습 fit()

시간 순 분할

```
prediction_length = 14 # 며칠 앞을 예측할 것인가

train_data, test_data = ts_data.train_test_split(
    prediction_length
)
```

모델 학습

```
predictor = TimeSeriesPredictor(
    prediction_length=14, # 14일 앞 예측
    target="target", # 예측 대상 = 대여량
    freq="D", # 일별 데이터
).fit(
    train_data,
    presets="medium_quality", # 성능·시간 균형
    time_limit=600, # 최대 10분
)
```

- 정형 데이터의 TabularPredictor와 같은 구조
- 시간 순 분할
미래 데이터가 학습에 유출되지 않도록 순서대로 나눔
- 평가 구간 = prediction_length = 14일
train_data 351일 · test_data 365일
(예측에 필요한 과거 구간 포함)
- presets · time_limit
성능과 학습 시간의 균형을 맞춤

시계열 예측 — 모델 학습

모델 성능 비교 leaderboard()

코드

```
predictor.leaderboard(test_data)
```

출력

model	score_test	score_val	fit_time
Chronos2	-0.1441	-0.2204	1.26
DynamicOptimizedTheta	-0.1764	-0.4337	0.01
Chronos2SmallFineTuned	-0.1783	-0.2113	85.48
AutoETS	-0.1802	-0.4058	0.01
WeightedEnsemble	-0.1896	-0.1407	0.79
DeepAR	-0.2418	-0.1590	45.50
SeasonalNaive	-0.2607	-0.5191	0.02
ChronosWithRegressor[bolt_small]	-0.2626	-0.1837	0.34
TemporalFusionTransformer	-0.2670	-0.1571	54.30
DirectTabular	-0.3233	-0.2106	6.22
RecursiveTabular	-0.6010	-0.4867	7.92

최상단이 1위 (score_test 기준)

- score_test — 평가용 성능, 0에 가까울수록 우수
- 1위 = Chronos2 (-0.1441), 사전학습 시계열 모델
- '클수록 우수'로 부호 통일 → 점수에 - 부착

앙상블이 늘 1위는 아니다

- WeightedEnsemble은 5위 (-0.1896)
- 강력한 단일 모델(Chronos2)이 앞설 수 있음

열 읽는 법

- score_val — 검증 성능 (모델 선택 기준)
- fit_time — 학습 시간(초)

시계열 예측 — 모델 학습

예측 predict()

코드

```
predictions = predictor.predict(train_data)

predictions.head()
```

출력

timestamp	mean	0.1	0.5	0.9
2018-11-17	18,858	8,962	18,858	22,339
2018-11-18	17,651	5,638	17,651	21,877
2018-11-19	17,611	5,467	17,611	21,884

정형 회귀와의 차이

- 정형: 예측값 하나
- 시계열: 평균 + 범위(분위수)

열 의미

- mean — 평균 예측(기댓값)
- 0.1 = P10 — 10%가 이보다 낮음
- 0.5 = P50 — 중앙값
- 0.9 = P90 — 90%가 이보다 낮음
- P10 ~ P90 = 80% 예측 구간
- 예: 11/17 평균 18,858대 · 80% 구간 8,962~22,339대

왜 범위인가

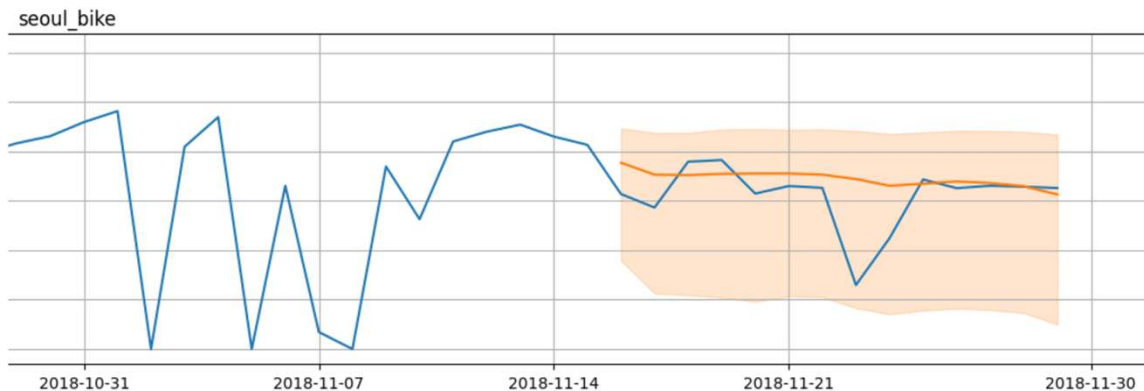
- 미래는 불확실 → 단일 값은 반드시 틀림
- 범위가 있어야 의사결정 가능

시계열 예측 — 모델 학습

예측 결과 시각화 plot()

코드

```
predictor.plot(  
    data=test_data,  
    predictions=predictions,  
    quantile_levels=[0.1, 0.9], # P10 ~ P90 표시  
    max_history_length=60,      # 과거 60일만  
);
```



그래프 구성

- 파란 선 — 과거 관측값 + 실제값 (Observed)
- 주황 선 — 평균 예측 (Forecast)
- 음영 영역 — P10 ~ P90 (80% 예측 범위)

확인할 점

- 예측선이 실제 흐름을 따라가는가?
- 실제값이 음영 안에 들어오는가?
- 음영이 오른쪽으로 갈수록 넓어지는가? (미래로 갈수록 불확실)

벗어난 날이 있다면

- 그날 무슨 일이 있었는가?
- 예: 2018-11-24 폭설 (적설 78cm)
- 대여량 16,314 → 6,477대로 급락
- 모델은 과거 대여량만 봤을 뿐, 눈이 올 줄 몰랐다

시계열 예측 — 모델 학습

연구자가 조절하는 학습 설정

```
predictor = TimeSeriesPredictor(  
    prediction_length=14,  
    target="target",  
    freq="D",  
    eval_metric="RMSE",          # 평가 기준 변경  
)  
.fit(  
    train_data,  
    presets="fast_training",  
    time_limit=180,
```

설정 표

설정	역할	값의 예
prediction_length	며칠 앞을 예측	14 (2주)
known_covariates_names	미래를 아는 변수	["seasons", "holiday"]
eval_metric	성능 판단 기준	WQL(기본) · RMSE · MASE
presets	성능·시간 균형	fast_training / medium_quality / high_quality / best_quality
time_limit	최대 학습 시간(초)	600 (10분)

시계열 예측 — 모델 학습

변수 중요도 feature_importance()

코드

```
predictor.feature_importance(  
    data=test_data,  
    time_limit=300,  
)
```

출력

feature	importance
functioning_day	0.0323
wind_speed	0.0162
rainfall	0.0061
humidity	0.0048
dew_point	0.0014
visibility	0.0013
temperature	-0.0018
solar_radiation	-0.0035

정형 데이터와 같은 원리

- 순열 중요도 — 값을 뒤섞어 성능 하락폭 측정
- 크게 떨어질수록 중요한 변수

확인할 점

- 날씨 변수가 상위에 오르는가?
- 값이 0에 가깝다 → 모델이 안 쓰는 변수

모델마다 쓰는 변수

모델	과거만 아는 값 (기온·강수량)
통계 (ETS·SeasonalNaive)	❌ 안 씀
트리 (RecursiveTabular)	❌ 안 씀
TFT · Chronos2	✅ 사용

실습 ② — 학습과 해석

데이터 선택

- 앞의 실습 데이터셋 중 하나 선택
- 실습①의 준비 · 변환 과정을 그 데이터에 적용

학습 · 해석

4 · 학습

`TimeSeriesPredictor(prediction_length=...).fit(train)` · leaderboard

5 · 해석

`predict` · `plot` 으로 예측 시각화
예측 구간(P10~P90) 확인

AutoGluon TimeSeries — 실습 코드

② 시계열 형식 변환

- 3열 형식(item_id · timestamp · target)을 **TimeSeriesDataFrame**으로 변환
- 결측 시점 보정 전 집계 방식 선택 필요 — 대여 건수는 합계, 기온·속도 등은 평균 적용

```
# 1) 3열 형식 → TimeSeriesDataFrame
from autogluon.timeseries import TimeSeriesDataFrame

ts = TimeSeriesDataFrame.from_data_frame(
    df, id_column="item_id", timestamp_column="timestamp",
)

# 2) 주기 통일 + 결측 시점 보정
ts = ts.convert_frequency(freq="h", agg_numeric="sum")
ts = ts.fill_missing_values()

# 3) 변환 결과 확인
print(ts.freq, ts.num_items, len(ts))
ts.head()
```

AutoGluon TimeSeries — 실습 코드

③ 예측 모델 학습과 평가

- 별도 split 없이 전체 **ts**를 그대로 투입 — 검증 구간 분리는 AutoGluon이 내부에서 자동 처리
- **prediction_length**는 예측하려는 시점 수 — 데이터 주기 기준으로 지정

```
# 1) 학습 (내부 검증 구간 자동 분리)
from autogluon.timeseries import TimeSeriesPredictor

predictor = TimeSeriesPredictor(
    prediction_length=24,          # 향후 24시간 예측
    target="target", eval_metric="MASE",
).fit(
    train_data=ts,                # 전체 데이터 그대로
    time_limit=600,               # 초 단위 탐색 예산
    presets="medium_quality",
)

# 2) 모델 비교 및 예측 - 내부 검증 점수 기준
predictor.leaderboard()
pred = predictor.predict(ts)      # 시점별 평균·분위수 예측
```